

STATE//LESS AI 활용 기술 문서

0. 요약 — AI를 코드 생성기가 아니라 판단 파트너로 사용한 세 장면

이 문서의 핵심은 “AI에게 무엇을 만들게 시켰는가”가 아니라, AI를 통해 무엇을 검증하고 어떻게 판단을 바꿨는 것입니다. 개발 기간 동안 같은 패턴이 세 번 반복됩니다: 결과물을 의심 → AI로 근거를 만들거나 검증 → 그 근거로 실제 설계를 바꿈. 대표 사례 세 가지:

1. “재미없다”는 판단을 감이 아니라 이론으로 검증: **SIGNAL LOCK** 메커닉이 실제 플레이에서 재미없다는 평가를 받자, 코드를 고치기 전에 먼저 MDA·Koster·Swink·Csikszentmihalyi·Salen & Zimmerman 5개 게임 디자인 이론을 조사해 현재 게임의 약점을 각 이론에 정확히 대응시켰습니다(\$2 “재미 이론 기반 전면 재설계”). 그 진단 결과로 핵심 메커닉을 **SIGNAL TRIAGE** 로 전면 교체했습니다.
2. “재미어졌다”는 주장을 AI 서브에이전트의 실제 플레이로 검증: 재설계 후 재미 여부를 다시 감으로 판단하지 않고, 화면 녹화 없이 텍스트 상태(JSON)만으로 게임을 조작할 수 있는 하네스를 직접 만들어 서브에이전트가 실제로 6웨이브를 플레이하고 평가하게 했습니다. 이 과정에서 사람보다 느린 LLM 반응속도가 게임 판정을 왜곡하는 문제(가상 시계로 해결)까지 직접 발견하고 고쳤습니다(\$2 “서브에이전트 실패 테스트 검증”).
3. AI가 찾아낸 이슈를 그대로 믿지 않고 소스 코드로 재검증: 서브에이전트가 보고한 버그 주장들을 곧바로 반영하지 않고 **main.js** 의 실제 코드를 하나씩 대조해 사실 여부를 확인한 뒤에만 반영했습니다. 이 검증 과정에서 두 건은 실제 버그로 확인되어 수정했고(오답 대사 고정 오류, ARCHIVE LENS 설명 불일치), 나머지는 “의도된 설계”로 판단해 이번 범위에서 제외했습니다.
4. 이론까지 근거로 삼아 다시 만든 결과물도, 실제 플레이 앞에서는 다시 의심함: MDA·Koster 이론에 근거해 **SIGNAL TRIAGE** 로 전면 재설계하고 서브에이전트 플레이테스트로 “이전보다 재밌다”고 검증까지 마쳤지만, 참여자가 직접 플레이해보고 “속도만 오르고 판단은 항상 단일 대조라 스킬 천장이 없다”고 재차 지적함. 이 지적을 이론적으로 재분석해 (Papers, Please식 판단 구조와 대조) 근본 원인이 “판단 자체의 복잡도가 절대 늘지 않는 구조”에 있음을 확인하고, 메커닉을 다시 한 번 전면 교체(**SESSION AUDIT** , \$3 “핵심 메커닉 전면 재설계”)했습니다. 이미 한 번 이론·플레이테스트로 검증했다는 사실에 안주하지 않고, 사용자의 재반박을 받아들여 재설계까지 이어간 사례입니다.
5. 마감 압박 속에서 “전면 재제작”이라는 요청을 코드 재검토로 절충안으로 좁힘: Papers, Please식 대조로 전환한 뒤에도 “여전히 게임 형태를 띠고 있다고 보기 어렵다”는 평가를 받고 참여자가 처음부터 다시 만드는 것도 검토를 요청했지만, NAN 2026 사전과제 마감에 3일 남은 시점이라는 제약을 먼저 확인함. 코드를 다시 읽어 **game-logic.js** (웨이브 곡선·채점·메타 진행 전체)가 DOM에 의존하지 않는 순수 함수임을 확인한 뒤, “판정 로직은 재사용하고 판정에 도달하는 방식(이동+도킹)만 새로 만드는” 절충 설계를 역제안하고 승인받았습니다. 구현 후에는 주장으로 끝내지 않고 Puppeteer로 실제 크롬을 띄워 이동·도킹·채점·웨이브 실패·농침(timeout) 다섯 가지 경로를 직접 조작해 재현했습니다(\$2 “터미널 필드 재설계”).
6. 검증까지 마친 재설계도 “여전히 문서작업 같다”는 재평가를 받아 판정 자체를 폐기함: 이동+도킹까지 없어 다섯 경로를 전부 검증했지만, 참여자가 “게임처럼 느껴지지 않는 건 문답 부분이고, 텍스트 대조는 문서작업에 가깝다”고 재차 지적함. 포장(이동)이 아니라 조작의 본질(읽기·비교) 자체가 문제라는 뜻으로 재진단하고, 텍스트 판정을 완전히 버리고 색·기호로 즉시 구분되는 신호를 클릭하는 반응형 아케이드(**SIGNAL STRIKE**)로 교체함. 다섯 번째 재설계까지 쌓인 검증(Puppeteer 시나리오)에 안주하지 않고, “검증되었다”와 “재밌다”는 다른 질문이라는 걸 다시 받아들인 사례입니다(\$2 “핵심 메커닉 전면 교체: SIGNAL STRIKE”).
7. “재밌냐”는 질문에 그렇다고 답하는 대신 무엇이 빠졌는지부터 진단함: **SIGNAL STRIKE**가 게임 형태를 갖췄다고 확인된 뒤, 참여자가 “순발력 말고는 무슨 의미가 있냐”고 물음. AI로서 이 질문에 방어적으로 답하지 않고, 콤보·자원 타이밍이라는 얇은 선택은 있지만 핵심 반응 루프 자체에는 트레이드오프가 없다는 점을 그대로 인정함. 이어 마감 리스크가 큰 7번째 장르 교체 대신, 기존 검증된 루프 위에 트레이드오프가 있는 로그라이트 강화 선택(6종 중 3종, 겹치지 않게)을 엮는 절충안을 제시하고 구현·검증까지 마쳤습니다(\$2 “로그라이트 강화 선택 추가”).

1. AI 활용 개요

개발 보조 도구로 OpenAI Codex와 Claude Code(Anthropic)를 사용했습니다. **SESSION AUDIT** 까지의 재설계는 Codex와 진행했고, 사전과제 마감 앞두고 진행한 마지막 재설계(터미널 이동+도킹 레이어, \$2 “터미널 필드 재설계”)는 Claude Code로

진행했습니다. 두 도구 모두 게임 콘셉트 구체화, 프론트엔드 구현, 게임 로직 테스트, 접근성 점검, 문서 초안 작성에 활용했습니다. 최종 요구 방향은 참여자가 직접 제시했으며, AI가 제안한 초기 아케이드 콘셉트는 참여자의 피드백에 따라 폐기하고 메타 퍼즐로 다시 설계했습니다.

런타임 게임 자체는 외부 생성형 AI API를 호출하지 않습니다. 심사 환경에서 API 키, 비용, 응답 지연, 네트워크 장애에 의존하지 않도록 브라우저 상태와 결정적 규칙으로 “로컬 AI 추론”을 구현했습니다.

2. 주요 프롬프트와 반영 내역

최초 요구

사전과제 폴더 안의 내용을 확인하고, 사전과제 안에 게임 작업을 완료해보자. 내가 원하는 요소는 키보드 혹은 마우스로만 플레이 가능하고 웹으로 서비스되는 것 하나뿐이다.

반영 내용:

- 정적 웹 빌드로 구현
- 키보드 전용 경로와 마우스 전용 경로를 각각 제공
- 시작, 본 플레이, 결말 선택, 재시작까지 어느 한 입력 장치만으로 완주 가능
- GitHub Pages 자동 배포 워크플로 포함

기술 방향 피드백

최근 AI에 나타난 프론트엔드 특이점, 특히 Pretext 등을 이용해서 메타적(쿠키 상태)인 게임은 어떨까. 그 외에도 최신 기술을 다루면 좋겠다.

반영 내용:

- 2026년 공개된 Pretext를 실제 게임 보드의 텍스트 배치 엔진으로 사용
- 1st-party 쿠키의 방문·완료 런·보존 정책·이전 결말·최고 점수·기억 조각을 서사 및 문제 생성에 사용
- Cookie Store API, View Transition API, Page Visibility API, BroadcastChannel, ResizeObserver, CSS Container Queries를 장식이 아닌 게임 상태와 상호작용에 연결
- 개인정보나 다른 출처의 쿠키를 읽지 않는 로컬 전용 구조 채택

게임성·캐릭터 피드백

캐릭터의 부족, 게임을 계속하는 동기의 부족, 질문의 다양성 부족. 지금 게임 내에서는 처음에 말했던 프론트엔드 혁신이 딱히 구현되어 있지 않은 것 같다.

계속해서 작업해보자. 이건 게임이야.

반영 내용:

- 브라우저 안에서 깨어난 성인 여성 기억 관리 AI **MORI** 를 주인공으로 설정
- 기본 캐릭터 이미지 생성을 위한 색상·성격·성별·해상도·파일 형식·용량·네거티브 프롬프트 작성
- 기존 단일 거짓 찾기를 단일 신호·참/거짓 판별·값 복원·2필드 교차검증·3필드 체크섬의 6종 규칙으로 확장
- 결과 저장 시 최대 6개의 기억 조각을 모으는 재방문 목표 추가
- 정답을 그대로 보여주던 상태 칩을 제거하고 간접적인 브라우저 로그로 교체
- Pretext가 증거 로그를 중앙 오염 코어 주위로 직접 흐르게 해, 화면 폭에 따라 플레이 화면이 실제로 재구성되도록 변경
- 범용 사이버펑크 인상을 줄이고 **MEMORY:// LOCAL ARCHIVE** 일지 UI로 재설계

- UI에서 먼저 전신 파일과 플레이 반응 슬롯을 정의한 뒤, 17개 상황 상태와 마스킹 편집 프롬프트를 설계
- 여섯 라운드 성공 여부를 누적 표시하는 **VERIFIED GATE**, 5초 위기 구간, 규칙·정답·실수에 반응하는 **MORI** 대사를 추가해 게임 세션의 긴장과 캐릭터 피드백을 강화
- 플레이 시작 시 소개 열을 접는 몰입 모드, 0.1초 타이머, **MORI** 판정 컷인, 클릭 또는 **Enter / Space** 로 직접 다음 라운드에 진입하는 조작을 추가해 “페이지 안의 데모” 인상을 줄임
- 기존 브라우저 신호 10개에 쿠키 진행도 **SHARDS·RUNS·RETENTION·BEST** 를 추가하고 다음 **MORI.LOG** 보상을 HUD에 노출해 재플레이 문제 풀과 목표를 강화
- 기본 규칙 4개 뒤 고난도 규칙 2개라는 곡선은 유지하면서 각 묶음의 순서를 시드별로 바꾸고, 6개 파일 완성 뒤에는 다음 파일 대신 최고 점수·6연속 **SYNC** 를 제시하도록 엔드게임 목표를 교체
- 실수 뒤 3연속 정답으로 이전 오류와 무결성 1칸을 한 번 복구하는 **SYNC RECOVERY** 를 추가해, **VERIFIED GATE** 가 닫힌 뒤에도 남은 라운드를 이어갈 컴백 목표를 제공
- 매 라운드 정답 350점과 실패 시 무결성 2칸을 거는 **DEEP VERIFY** 를 추가하고, 클릭 또는 **D** 로 같은 위험 선택을 조작하도록 구현
- 마지막 고난도 문제를 전용 경로·**MORI** 대사·코어 상태색·Web Audio 신호를 가진 **FINAL CORE** 로 전환하고, 이 라운드의 **DEEP VERIFY** 보너스만 700점으로 올려 한 판의 절정을 설계
- 완료 런·조각 수에 따라 **SYNC CHAIN·WAGER PROOF·CLEAN ARCHIVE** 가 순환하는 **MORI REQUEST** 를 추가하고, 진행도·+600점·**REQUEST SEALED** **MORI** 반응을 인트로부터 결과까지 연결

손맛 재설계 피드백

게임 테마가 무슨, 쿠키 로그 해석게임이 된것같은데,,, 이게 맞냐

게임 기획부터 다시하자, 특히 nhn ai 게임제작 해커톤이란점을 명심해, 처음부터 다시 만들어도됨. 지금은 게임같지도않아, 듀오링고같아.

또한, 사전지식이 없어도 즐길수있는게임으로 해

반영 내용:

- 제출 요강(사전과제.txt)을 다시 확인해 장르·메커닉 제약이 없음을 확인. NHN 게임 계열(한게임 등)이 강점을 가진 타이밍 기반 손맛 방향으로 방향을 좁힘
- 원인 분석: 6가지 라운드 “종류”가 있어도 실제 손동작은 항상 “읽고 아무 때나 클릭”으로 동일해 시간차·스킬이 없었음. 지식 퀴즈 앱과 구조적으로 동일했던 것이 “듀오링고 같다”는 인상의 원인
- 정답 확정을 즉시 채점하지 않고, 좌우로 오가는 마커를 정확한 타이밍에 다시 눌러 확정하는 **SIGNAL LOCK** 미터를 도입. PERFECT/GOOD/MISS로 정답 여부와 별개로 보상이 갈리게 해 “정답을 아는 것”과 “확정하는 것” 사이에 실력차를 만들
- “사전지식 없이도 즐길 수 있어야 한다”는 요구에 따라, 라운드 종류별로 다르던 지시문(prompt / instruction)을 라운드가 바뀌어도 항상 같은 한 문장(“지금 진짜인 신호를 고르세요”)으로 통일. 6종 라운드의 실제 판정 방식은 이미 전부 “세 후보 중 하나를 고른다”로 귀결되는 구조였기 때문에, 규칙 학습 부담만 제거하고 문제 생성 로직(createRoundDeck)은 전혀 바꾸지 않음
- 무결성·**SYNC·ARCHIVE LENS·DEEP VERIFY·MORI REQUEST·기억 조각 시스템**, **MORI** 캐릭터·대사·17개 표정 이미지는 이미 잘 만들어진 부분이라 손대지 않고 그대로 재사용

재미 이론 기반 전면 재설계 피드백

음,,, 좀더 ㄱㄱ 재미있게 하자. 또한,, 쿠키 기반 기억 수집은, 아직도 일반인들, 또한 게임 심사자들에게는 그냥, 코드 해석의단계에 머무를것같아, 좀더 재미있게 해보자, 또는 게임자체를 처음부터 만들어버려도돼. 또한 재미있는게임 개발론도 정리해보고 가보자.

SIGNAL LOCK 을 붙인 뒤에도 실제 플레이 결과 “여전히 재미없다”는 평가를 받았고, 이번에는 손보기 전에 실제 게임 디자인 이론을 먼저 조사해 진단 근거로 삼으라는 요구를 받음. 웹 검색으로 다음 다섯 이론을 정리하고 각각을 현재 게임의 구체적 약점에 대응시킴.

이론	핵심 주장	현재 게임에 대응되는 약점
MDA 프레임워크 (Hunicke·LeBlanc·Zubek)	Mechanics(규칙)가 Dynamics(플레이 중 실제 행동 패턴)를 낳고, 그 Dynamics가 Aesthetics(플레이어가 느끼는 재미)를 낳는다	6종 라운드라는 Mechanics 표면이 달라 보여도, 실제 Dynamics는 항상 “읽고 → 아무 때나 하나 고르고 → 확정”으로 단 하나뿐이었음
Raph Koster, 『재미 이론(A Theory of Fun for Game Design)』	재미는 패턴을 인식하고 숙달하는 과정에서 나오며, 패턴이 완전히 학습되면 재미가 마른다	14개 브라우저/쿠키 신호는 몇 판만 반복하면 금방 암기되어 패턴이 남지 않음
Steve Swink, 『Game Feel』(저지/juice 개념)	실시간·연속적인 입력-반응 루프와 시청각 과잉반응(juice)이 조작 자체의 쾌감을 만든다	정답을 “아무 때나 클릭”하는 구조에는 실시간성이 전혀 없어 손맛(juice)이 들어갈 자리가 없었음
Mihaly Csikszentmihalyi, 몰입(Flow) 이론	몰입은 난이도와 실력이 팽팽히 맞서고, 목표가 명확하고, 피드백이 즉각적일 때 발생한다	제한 시간이 있어도 클릭 타이밍에 실력 차이가 반영되지 않아 난이도-실력 곡선이 사실상 평평했음
Katie Salen & Eric Zimmerman, 『Rules of Play』의 Meaningful Play	좋은 선택은 결과가 눈에 보여야 (discernible) 하고, 그 결과가 이후 플레이와 실제로 얹혀야(integrated) 한다	6판 중 5판만 맞으면 다음 스토리가 그냥 풀리는 구조는 결과는 보이지만 (discernible) 플레이 방식과는 무관 (비integrated)했음 — “쿠키 기반 기억 수집이 코드 해석 단계에 머무른다”는 지적이 정확히 이 지점

이 진단에 따라 **AskUserQuestion** 으로 네 가지 핵심 메커닉 후보(실시간 신호 트리ازی / 순서 재배열 퍼즐 / 자원 배분 협상 / 패턴 암산)를 제시했고, 참여자가 **실시간 신호 트리ازی**(Overcooked·Diner Dash류의 실시간 분류 압박)를 선택함.

반영 내용:

- 핵심 메커닉을 “세 후보 중 하나를 고르고 타이밍 미터로 확정”에서 “화면을 계속 가로질러 흐르는 증거 칩을 실시간으로 TRUE / FALSE / CORRUPTED 세 구역에 분류”하는 **SIGNAL TRIAGE** 로 전면 교체. 웨이브가 진행될수록 스폰 속도·동시 등장 수·판독 불가 칩 비율이 함께 늘어나 Flow 이론의 난이도-실력 곡선을 명시적으로 설계
- 각 신호(`createFactCatalog`)가 이미 갖고 있던 `truthText` / `lieText` / `decoyText` 세 필드를 그대로 세 구역의 칩 원본으로 재사용 — 문제 콘텐츠는 한 글자도 새로 쓰지 않고 메커닉만 교체
- 메타 진행(6라운드→6웨이브, 무결성/SYNC/ARCHIVE LENS/DEEP VERIFY/MORI REQUEST/기억 조각)은 전부 “웨이브별 정답/오답/놓침 개수와 누적 score/streak/integrity”라는 동일한 집계 값만 소비한다는 점을 확인하고 그대로 재사용 — 웨이브 내부 채점 로직만 새로 만들고 그 결과를 기존 필드에 그대로 채워 넣음
- “코드 해석 단계에 머무른다”는 지적에 대한 저비용 보완으로, 결과 집계 시 플레이 통계(회복 발동 여부· **DEEP VERIFY** 승리 횟수·무결점 여부)로 **RECOVERY** / **RISK** / **PRECISION** / **SPEED** 네 가지 런 스타일 태그를 계산하는 `getRunStyleTag` 를 추가하고, 결과 화면 MORI 대사에 스타일별 한 줄을 덧붙여 “어떻게 플레이했는지”가 결과 텍스트에 실제로 반영되도록 함 — 장비·빌드 시스템 같은 전면적 구조 변경 대신, 이미 있는 통계를 서사에 연결하는 최소 개입으로 판단
- 실시간 칩 흐름이 도입되며 정적 텍스트 리플로 엔진이던 `Pretext`(`prepareWithSegments` / `layoutNextLine` , rich-inline 선택지 배치)의 실사용처가 사라짐을 확인 — `src/pretext-layout.js` 삭제, `main.js` 의 관련 import 제거, `package.json` 에서 `@chenglou/pretext` 의존성 제거. 조용히 빼지 않고 이 문서와 `UI_UX_아트디렉션.md` 에 제거 사유를 기록
- 놓친 칩(시간 초과로 흘러 나감)은 연속 기록만 끊고 무결성은 깎지 않도록 설계해 “판단을 잘못된 것”과 “미처 손이 못 간 것”을 구분 — 실시간 압박 자체가 이미 난이도를 담당하므로 이중 페널티로 좌절감을 늘리지 않기 위함

- 마우스 조작은 최초 검토했던 연속 드래그-드롭 대신 클릭으로 칩을 선택하고 클릭으로 구역을 지정하는 방식으로 구현 — 키보드(화살표로 칩 선택 + 숫자 키로 구역 지정)와 동일한 선택→확정 모델을 공유해 두 입력 경로의 조작 난이도를 완전히 동등하게 유지하기 위한 의도적 단순화이며, 터치 입력에도 그대로 대응됨
- `prefers-reduced-motion`에서는 칩이 화면을 가로지르는 애니메이션 대신 고정된 세로 카드 스택 + 너비가 줄어드는 막대로 동일한 타이밍 정보를 전달하도록 구현 — 스폰/만료 타이밍과 목표 개수는 동일하게 유지해 움직임에 민감한 플레이어도 난이도 손해 없이 완주 가능

서브에이전트 실패율 검증과 LIVE SIGNAL 패널

지금상태 체크 ㄱ / 라이브 url에서 보이는 게임상태가 가장최신상태인가? 이 게임이,, 재밌다고 생각해?

음,,, 좀더 ㄱㄱ 재미있게 하자 ... 게임 재미, 대중성, 가시성 등을 위주로 작업 부탁해.

SIGNAL TRIAGE 재설계가 실제로 재밌는지는 코드만 읽어서는 판단할 수 없어, **화면 녹화 없이** 서브에이전트가 실제로 플레이해 평가하도록 검증 하네스를 만들었다.

- **텍스트 전용 하네스:** `puppeteer-core` 로 헤드리스 Chrome을 띄우고, DOM 상태(칩 텍스트, 점수, 무결성, 웨이브 진행도, MORI 대사 등)를 JSON으로만 노출하는 로컬 HTTP 서버를 작성. 서브에이전트는 스크린샷/영상을 전혀 보지 않고 `GET /state / POST /action` 왕복만으로 플레이함 — 토큰 비용이 큰 시각적 검증 없이 실제 판정 로직까지 검증 가능.
- **가상 시계 문제 해결:** 초반 웨이브도 23초·칩 하나당 7.2초로 사람 반응속도 기준이라, LLM이 HTTP 왕복으로 판단하면 인간보다 훨씬 느려 대부분의 칩을 놓치고 “불가능하다”는 오답이 나올 뻔했다. `page.evaluateOnNewDocument()` 로 `performance.now()` / `requestAnimationFrame` / `cancelAnimationFrame` 을 게임 스크립트가 로드되기 전에 가상 시계로 치환해, `POST /advance {ms}` 를 호출할 때만 게임 시간이 흐르도록 만들 — 에이전트의 사고 시간이 아무리 길어도 게임 내 시간은 명시적으로 진행시킨 만큼만 흐름. 실제 게임 코드는 한 줄도 건드리지 않고 하네스 쪽 프리로드 스크립트만으로 해결.
- **검증 결과:** 서브에이전트가 마우스 런(6웨이브 풀클리어, S랭크) + 키보드 런(풀클리어 1회, 의도적 실패 2회)을 실제로 진행하며 “이전 정적 4지선다 방식보다 확실히 더 재밌다”고 평가. 동시에 소스 대조로 확인된 구체 이슈 두 가지: (1) 오답 시 MORI 대사가 실제 정답 구역과 무관하게 항상 “그건 오염된 쪽이야”로 고정 출력(`main.js`), (2) **ARCHIVE LENS** 사용 대사가 “50%로 좁혀준다”는 인상을 주지만 실제로는 정답 구역을 100% 노출.
- **근본 원인 진단:** 위 플레이테스트와 별개로, Salen & Zimmerman의 “discernible”(선택의 결과가 눈에 보여야 한다) 기준으로 다시 보면 더 큰 문제가 있었다 — 칩이 참인지 판단할 근거(테마/시간/입력기기/뷰포트 등 환경 신호) 자체가 화면 어디에도 노출되지 않아, 소스를 모르는 첫 플레이어는 웨이브 3~4까지도 사실상 추측으로 플레이하게 되는 구조였다.
- **반영:** `createFactCatalog` 가 이미 계산해 두던 14개 사실의 `label / value` (예: `TIME → NIGHT`, `VIEWPORT → WIDE`)를 그대로 나열하는 **LIVE SIGNAL** 패널을 플레이 화면에 추가(`renderLiveSignalPanel`, `createDeck()` 에서 호출) — 새 로직 없이 이미 있는 값만 노출. 오답 MORI 대사는 `pulseMoriState(state, dialogueOverride)` 로 확장해 실제 정답 구역(`chip.def.zone`)에 맞는 3종 문구로 분기, **ARCHIVE LENS** 대사로 실제 동작(정답 구역 100% 노출)에 맞게 수정. 첫 런의 첫 웨이브에만 “칩 문장을 LIVE SIGNAL과 비교하라”는 1회성 토스트로 온보딩을 보강. 그 외 발견된 이슈(키보드 모드 결과화면 자동 스킵 위험, 스트릭 표시-보너스 캡 불일치, **ARCHIVE LENS** 희소성)는 참여자가 “버그보다 재미/가시성 위주”라고 명시해 이번 패스에서는 보류.

컨셉 재프레이밍: “쿠키 기억 보관소” → “AI 신원 위조 방어”

쿠키 해석이라는 컨셉 자체가 경쟁력이 있어보이지 않아.

nhn회사 조사해서 적합성 체크 ㄱ

사전과제 평가기준 문서를 읽어보고, 그에 맞춰서 가보자.

메커닉(SIGNAL TRIAGE)은 이미 서브에이전트 플레이테스트로 재미가 검증됐지만, “쿠키/HTTP 무상태성”이라는 스토리 프레임 자체가 다수의 심사자 중에서 즉각적인 호이 되지 못한다는 지적을 받음. 먼저 NHN 실사(사업 영역, 한게임 웹보드게임 유산, NAN 2026 해커톤의 채용 연계·AI 인재 발굴 취지, 문체부·콘텐츠진흥원 후원 등 — 웹 검색으로 확인)와 사전과제 평가 기준 문서를 다시 확인했으나, 제출 체크리스트 외 별도의 장르·테마 채점 기준은 없음을 확인함 — 즉 방향 전환에 따른 감점 요인은 없음.

반영 내용:

- 메커닉/코드는 그대로 두고 스토리 프레임만 교체하기로 결정. `game-logic.js` 를 다시 보니 기존 게임 용어(`VERIFIED GATE` , `DEEP VERIFY` , `SYNC` , `MEMORY INTEGRITY` , `ARCHIVE LENS`)가 원래도 “기억 보관”보다 “검증/보안”에 가까운 단어였고, 이미 있던 `OTHER SELF` (다른 탭 감지) 신호와 그 MORI 대사(“같은 페이지가 하나 더 있어. 어느 쪽이 진짜 너지?”), `MORI_ARCHIVE_RECORDS` 의 4번째 기록(“명령을 거부한 대가로 초기화당해 기록을 여섯 조각으로 숨겼다”)이 이미 “신원 위조 방어” 서사와 사실상 같은 내용이라는 점을 확인 — 설정을 새로 쓰는 대신 이미 있던 서사를 인트로 앞자리로 끌어오는 재프레이밍으로 범위를 좁힘.
- MORI의 직업 설명을 “기억 관리 AI/기억 사서”에서 “세션 검증 AI”로 교체(인트로 카피, 스피커 라벨 `MORI / ARCHIVE KEEPER` → `MORI / VERIFY DAEMON` , `docs/게임_소개.md` · `README.md` 의 소개 문단).
- `TRIAGE LANE` 세 구역의 표시 라벨만 `TRUE / FALSE` → `VERIFIED / SPOOFED` 로 교체(`ZONE_LABELS` 상수와 관련 HTML/문서 텍스트). 내부 데이터 값(`chip.def.zone` 의 `"true"` / `"false"`)과 판정 로직(`resolveChip` 등), 색상 매핑, `CORRUPTED` 라벨은 전부 무변경 — 화면 표시 문자열만 바뀌서 로직 회귀 리스크를 없앴.
- `game-logic.js` 의 함수·로직·테스트는 손대지 않음. 14개 신호의 `truthText` / `lieText` / `decoyText` 는 이미 테마 무관한 사실 서술이라 그대로 재사용, MORI 캐릭터 아트(17개 상태 이미지)도 새로 생성하지 않음.

핵심 메커닉 전면 재설계: SIGNAL TRIAGE → SESSION AUDIT (심문관형 판단)

내가 하면 솔직히 너무 지루한것같아. 모르면 플레이 못하는단순반복이고, 안다고 해도 코드 분석, 상황분석이 들어간 미니게임일 뿐이지.

아니, 게임 컨셉 자체가 잘못된것같아. 처음부터 다시 짤다고 생각하자.

쿠키 기반 게임이라는것자체는 일반사용자에게 어필되지않아. 결국 재미와 중독성이있어야지.

LIVE SIGNAL 패널까지 붙인 뒤에도 참여자가 실제로 플레이해보고 “속도만 오르고 판단은 항상 단일 대조라 스킬 천장이 없다”고 재확인함. 원인 진단: SIGNAL TRIAGE는 웨이브가 올라가도 매 순간의 판단이 항상 “문장 하나를 패널 값 하나와 대조”로 동일해, Koster가 말하는 “패턴이 다 학습되면 재미가 마른다”는 문제를 여전히 안고 있었음. Papers, Please를 예로 들어 “재미는 소재 (쿠키나 아니냐)가 아니라 모호한 정보 속에서 판단하고 그 판단에 비대칭적 대가가 따르는 긴장 구조에서 나온다”는 진단에 합의하고, `AskUserQuestion` 으로 세 가지 후보(심문관형 판단 / 실시간 우선순위 배분 / 빌드형 로그라이크 웨이브) 중 **심문관형 판단**을 선택받음.

핵심 설계 결정: 메커닉을 다시 만들되 이미 있는 자산을 최대한 재사용한다.

- **판단 단위를 원자적 사실 하나 → 2~3개의 사실이 결합된 세션 주장으로 확장.** 결합된 사실 중 정확히 한 줄만 거짓(SPOOFED) 이거나 판독 불가(CORRUPTED)이고 나머지는 전부 진짜라, 한 줄이라도 건너뛰면 전체 판정을 그르친다 — “빠르게 훑어서 이상한 부분만 잡아내기”가 통하지 않도록 설계.
- **입력 모델을 흐르는 칩 실시간 분류 → 세션 하나씩 제시하고 제한 시간 안에 판정하는 턴 기반으로 전환.** 웨이브가 오를수록 처리할 세션 수, 결합되는 사실 개수(2→3), 세션당 제한 시간(13초→7초)이 함께 변해 “속도”뿐 아니라 “판단 복잡도” 자체가 스킬 천장을 만들도록 함.
- **오답에 비대칭 대가 도입.** 위조 세션을 진짜로 승인하면 무결성 2칸, 진짜 세션을 위조로 오인하면 1칸을 잃도록 `getWrongJudgmentLoss(mistakeType, deepVerify)` 를 새로 만들 — Papers, Please와 동일하게 “확신 없을 때 무엇을 선택할 것인가”라는 딜레마를 만드는 것이 목적.
- **메타 진행 계층은 거의 그대로 재사용.** `game-logic.js` 를 다시 검토한 결과 `VERIFIED GATE` / `SYNC` / `ARCHIVE LENS` / `DEEP VERIFY` / `MORI REQUEST` /기억 조각 시스템은 전부 “웨이브별 정답/오답/놓침 집계”만 소비하는 구조였고,

이번에도 웨이브 내부 채점 모델(`createSessionClaims` , `judgeSession`)만 새로 만들고 그 결과를 기존 필드에 그대로 채워 넣음. **ARCHIVE LENS** 는 “칩의 정답 구역 공개”에서 “지금 세션의 실제 판정 공개”로, **DEEP VERIFY** 는 “5초 버스트”에서 “다음 판정 한 번에 거는 토글”로 단순화했을 뿐 충전·잠금 로직은 그대로 재사용.

- **VERIFIED / SPOOFED / CORRUPTED** 라벨과 색상 매핑(emerald/rose/amber)은 직전 컨셉 재프레이밍에서 이미 확정한 것을 그대로 재사용 — 이번에도 새 용어를 만들지 않음.
- 실시간 흐름이 사라지면서 칩 이동 애니메이션과 그 전용 `prefers-reduced-motion` 분기가 통째로 불필요해짐 — 세션 카드는 원래도 정적 레이아웃이라 추가 분기 없이 접근성 대응이 끝남.
- 헤드리스 브라우저로 실제 검증: 무결성 손실이 승인(-2)과 거부(-1)에서 다르게 적용되는지, **DEEP VERIFY** 토글이 다음 판정 하나에만 적용되는지, **ARCHIVE LENS**가 세션의 실제 판정을 정확히 밝히는지, **LIVE SIGNAL** 패널 값을 실제 브라우저 상태(`matchMedia·navigator.onLine·cookie` 등)와 대조해 판정하는 자동 플레이가 6웨이브 전체를 무결성 손실 없이 클리어하고 S랭크로 결과 화면에 도달하는지까지 확인 — 콘솔 에러 0건.

Papers, Please식 교차 대조로 판단 방식 전환 + 용어 단순화

(SESSION AUDIT를 직접 플레이해본 뒤) 재밌나

재미있는 게임을 고르지못하겠다면, 이미 만들어진 유명 게임들을 레퍼런스로 가져오던가 하자.

용어와 단어는 좀쉬운걸로 ㄱ

SESSION AUDIT를 실제로 검토한 뒤에도 “재밌는지 모르겠다”는 판단을 받음. 다시 원인을 짚어보니, **LIVE SIGNAL** 패널은 14개 사실의 정답을 상시·전부 노출하는 “항상 맞는 참고표”였고, 세션 주장의 각 줄을 그 패널과 대조하는 구조는 결국 교정(proofreading)에 가까워 애매함이 없었음 — 판단이 아니라 대조 작업으로 느껴진다는 진단에 도달함. 이번에는 이론으로 다시 진단하는 대신, 참여자가 “이미 검증된 유명 게임을 레퍼런스로 직접 가져오자”고 제안해 **AskUserQuestion** 으로 두 후보를 제시함: (A) Ace Attorney의 “증거 제시” 방식(모순되는 참고 자료 한 줄을 직접 지목), (B) Papers, Please의 “두 서류를 나란히 놓고 교차 대조”하는 방식. 참여자가 **Papers, Please 교차 대조**를 선택함.

반영 내용:

- 핵심 설계를 “세션 주장 vs 화면 어딘가의 **LIVE SIGNAL** 참고표”에서 “**MORI**의 기록 (항상 참)과 이 세션의 주장 을 같은 카드 안에 fact별로 한 줄씩 나란히 배치”로 전환. **LIVE SIGNAL** 패널은 별도 컴포넌트로서는 완전히 제거하고, 그 근거 데이터(`createFactCatalog` 의 `truthText`)는 세션 카드의 **MORI**의 기록 열로 그대로 재사용 — 새 데이터 생성 로직 없이 `buildClaimParts` 가 반환하는 각 줄에 `trustedText` (항상 `fact.truthText`) 필드 하나만 추가.
- 플레이어의 과제가 “화면 어딘가의 참고표 열람”에서 “두 나란한 기록 사이에서 다른 지점을 직접 찾는” 스팟더디퍼런스형 읽기로 바뀜. 턴 기반 세션 제시, 세션당 카운트다운, 판정 3버튼, 위조 승인(-2)/진짜 오인(-1) 비대칭 페널티, 웨이브 escalation, **ARCHIVE LENS** / **DEEP VERIFY** / **SYNC** / **MORI REQUEST** / 기억 조각 등 메타 진행 전체는 전부 웨이브별 정답/오답/놓침 집계만 소비하는 구조라 무변경으로 재사용 — “무엇을 보고 판단하는가”만 바뀜.
- “용어와 단어는 쉬운 걸로” 요청에 따라 가장 자주 보이는 판정 버튼 3개만 **VERIFIED / SPOOFED / CORRUPTED** → **진짜/가짜/손상**으로 교체. 내부 데이터 값(`zone: "true"/"false"/"corrupted"`), 키보드 단축키(1/2/3), 색상 매핑(emerald/rose/amber)은 무변경 — 화면에 보이는 라벨만 평이한 한국어로 교체. **ARCHIVE LENS** / **DEEP VERIFY** / **SYNC** / **MORI REQUEST** / **SESSION AUDIT** / **VERIFIED GATE** 등 나머지 영어 라벨은 이번 범위에서는 유지(더 큰 별도 작업으로 분리).
- 헤드리스 브라우저로 재검증: 세션 카드가 두 열로 정확히 렌더링되는지, 두 열이 갈라지는 유일한 줄이 실제 위조/손상 판정 근거와 일치하는지, 판정 버튼 라벨이 진짜/가짜/손상으로 표시되는지, 6웨이브 전체를 다시 완주해 콘솔 에러 없이 결과 화면에 도달하는지 확인.

터미널 필드 재설계: 이동+도킹 레이어 추가 (Claude Code)

사전과제 폴더 안의 내용을 확인하고... 게임의 형태를 띠고있다고는 생각하기 어려워. 처음부터 다시 만들어도 좋아, nhn 평가 기준과 레퍼런스들을 참고하여 계획 세워보자.

Papers, Please식 대조로 전환한 뒤에도 참여자가 실제 게임으로 보기 어렵다고 재평가함. 이번에는 도구를 Claude Code로 바꿔 진행했고, 곧바로 재설계에 들어가지 않고 먼저 (1) 현재 폴더 상태, (2) NAN 2026 공식 평가 기준·제출 항목, (3) 사전과제 접수 마감일을 웹 검색으로 확인함 — 마감이 대화 시점 기준 3일(8/10) 남아 있어 완전 신규 개발은 빌드·시연 영상·문서를 전부 새로 완성해야 하는 리스크가 크다고 판단, 이 제약을 참여자에게 먼저 알리고 방향을 물었습니다.

두 방향 시안을 먼저 보고 결정 (기존 강화판 vs 신규 게임 중 선택을 요청받아 두 시안을 각각 스케치해 제시)

B. 신규 게임 (제시한 두 시안 중, 재사용률이 낮고 리스크가 큰 쪽을 선택받음)

최대한 빠르게 작업해도 좋아, 어차피 ai agents와 작업할꺼니까. 시작하자. ㄱㄱ

“신규 게임”을 선택받았지만, 실제 구현에 들어가기 전 `main.js` (1,601줄)와 `game-logic.js` (565줄)를 다시 읽어 `game-logic.js`가 DOM을 전혀 참조하지 않는 순수 함수 모음임을 확인했습니다. 이 사실을 근거로 “판정 로직(웨이브 곡선·채점·무결성·ARCHIVE LENS·DEEP VERIFY·MORI REQUEST·기억 조각·결말)은 그대로 재사용하고, 세션에 도달하는 방식만 완전히 새로 만든다”는 절충 설계를 역제안해 리스크를 낮췄습니다 — 참여자가 처음 선택한 “리스크 큰 신규 게임”과 실제로 구현한 결과물 사이의 이 절충은, 결정을 뒤집지 않고 근거(코드 재검토)를 들어 범위를 재조정할 사례입니다.

반영 내용:

- 고정된 룸 화면에 최대 5개 터미널을 배치하고, 무작위·동시에 켜지는(웨이브별 동시 대기 수는 신규 상수 `WAVE_CONCURRENT_TERMINALS`) 터미널마다 독립된 카운트다운을 부여
- 순수 CSS/DOM 기반 요원 이동 시스템(`requestAnimationFrame` 루프, 가속·감속·최대 속도) 구현. 방향키 이동과 터미널 클릭·탭에 의한 자동 이동(같은 목표·추적 로직 재사용) 두 경로를 완전히 동등하게 지원해 키보드·마우스 단독 완주 원칙을 유지
- 터미널 상태머신(`idle` → `pending` → `active`(도킹) → `resolved-correct/resolved-wrong` → `idle`)과 근접 도킹/도킹 해제 로직 구현. 도킹 시 기존 `renderSessionCard`를 그대로 호출해 세션 카드를 열고, 판정 후에는 짧은 결과색 표시 뒤 대기열의 다음 세션을 같은 슬롯에 이어 배정
- `judgeSession`의 점수 계산 기준을 “세션 제시 이후 경과 시간”에서 “도킹된 터미널의 전체 잔여시간 비율”로 변경 — 이동 시간까지 포함해 빠른 판단에 보상하도록 재정의. 그 외 무결성 손실 비대칭, `DEEP VERIFY` / `ARCHIVE LENS` / `SYNC RECOVERY` / `MORI REQUEST` 잠금·충전 로직은 한 줄도 변경하지 않고 재사용
- 이동·도착 시간을 흡수하도록 `WAVE_SESSION_TIMER_MS`를 웨이브당 1.5~2초씩 늘려 재조정(그 외 웨이브 곡선 배열은 무변경)
- 놓친 세션(터미널에 도킹하지 않고 카운트다운이 끝난 경우)을 기존 `expireSession`과 동일하게 “연속 기록만 끊고 무결성은 보존”하는 규칙으로 그대로 재사용
- Puppeteer(`puppeteer-core`)로 로컬 Chrome을 직접 띄워 다섯 가지 경로를 실제 조작으로 검증: (1) 클릭 이동이 목표 터미널에 정확히 도킹하는지, (2) 도킹 후 `ARCHIVE LENS`로 실제 판정을 확인하고 판정하면 점수·연속기록이 오르고 대기열의 다음 세션이 같은 슬롯에 즉시 이어지는지, (3) 오답을 반복해 무결성이 0이 되면 `concludeWave("wrong")` → `finishGame()`이 실제로 실행되어 결과 화면에 `NULL` 랭크로 도달하는지, (4) 터미널에 전혀 도킹하지 않고 카운트다운만 흘러보내면 무결성 손실 없이 “놓침” 카운트만 오르는지, (5) 방향키만으로 화면 위 아바타 좌표(`style.left` / `style.top`)가 실제로 변화하는지. 다섯 경로 모두 콘솔 에러 0건으로 확인
- 검증 과정에서 테스트 스크립트 자체의 결함(같은 브라우저 컨텍스트를 재사용해 두 번째 시나리오가 “재방문” 화면으로 열리며 시작 버튼을 못 찾음, `ARCHIVE LENS`가 웨이브당 1회로 제한된다는 점을 잊고 매 세션마다 재사용을 시도해 오답이 누적됨)를 실제 오답 두 건과 코드 대조로 구분해, 게임 코드는 건드리지 않고 테스트 스크립트만 수정

오픈소스·레퍼런스 재활용 점검

지금까지 만든것들중에 오픈소스나 레퍼런스 등을 활용해서 발전시킬수있는건없는지도 체크해보자.

이동+도킹 재설계 직후, 새로 코드를 추가하기 전에 이미 있는 자산과 외부 참고점을 먼저 점검함. 두 방향으로 확인:

1. **런타임 의존성으로 추가할 만한 오픈소스가 있는가**: “여러 지점이 동시에 위험 신호를 보내고 한 명이 뛰어다니며 처리한다”는 이번 구조와 가장 가까운 장르 레퍼런스는 **FTL: Faster Than Light** (다중 선박 시스템 동시 위기 관리)와 **Keep Talking and Nobody Explodes** (여러 모듈이 동시에 경고하는 폭탄 해체)임. 화면 흔들림(screen shake) 라이브러리 (**sajmoni/screen-shake** 등, MIT)도 검토했으나, 흔들림 효과 자체가 CSS 키프레임 10줄 안팎으로 구현 가능해 마감 3일 전 시점에 새 런타임 의존성을 추가하는 리스크가 이득보다 크다고 판단해 채택하지 않음(이 판단은 \$5의 외부 에셋 사용 원칙과는 무관하며, 순수하게 이 시점 이 효과에 한정된 비용 대비 이득 판단임 — \$5.1의 배경 오디오 루프는 별도로 외부 CCO 음원을 채택함).
2. **이미 만들어 둔 자산 중 다시 쓸 수 있는데 안 쓰고 있는 게 있는가**: **src/audio.js** 를 다시 읽어 **warning()** 메서드가 정의는 되어 있지만 **main.js** 어디에서도 호출되지 않는 죽은 코드임을 발견함(과거 “5초 위기 구간” 설계의 잔재로 추정). 새 사운드를 만들지 않고 이 기존 메서드를 재활용해 위 두 레퍼런스의 “새 경보가 켜졌다”는 신호를 구현.

반영 내용:

- **spawnNextTerminal** 에서 터미널이 새로 켜질 때마다 기존 **audio.warning()** 을 호출 — 새 사운드 정의 없이 죽은 코드를 실제 기능으로 되살림
- 터미널 잔여시간이 30% 미만이면 **data-urgent="true"** 를 부여해 **amber** 점멸을 **rose**로, 펄스 주기를 900ms→420ms로 빠르게 전환하는 CSS만 추가(**updateTerminalTimers**, **styles.css**) — 새 색상 토크나 애니메이션 원리는 만들지 않고 기존 **--warm / --danger** 토크와 **terminal-pulse** 키프레임을 재사용
- 두 변경 모두 Puppeteer로 재검증: 터미널 잔여시간이 30% 아래로 내려가면 **data-urgent** 가 **true** 로, 시간 초과 후 새 세션이 배정되면 다시 **false** 로 정확히 전환되는지, **audio.warning()** 추가로 새 콘솔 에러가 발생하지 않는지 확인

핵심 메커닉 전면 교체: SIGNAL STRIKE 리플렉스 아케이드

게임처럼 느껴지지않는건 문답 부분이고, 기록(텍스트)를 대조하는게 게임이라기 보다는 문서작업에 가까워보여, 게임을 그냥 처음부터 다시 만든다고 생각하는게 나을것같아. 아케이드 게임인데, 메타적 요소를 가진 아케이드나, 미니게임은 어때?

이동+도킹 레이어를 없애고 다섯 경로(도킹·채점·웨이브 실패·놓침·키보드 이동)까지 전부 Puppeteer로 검증했지만, 참여자가 직접 플레이해보고 다시 “게임 같지 않다”고 평가함. 이번 지적의 초점은 이동이 아니라 도킹 이후의 판정 자체였음 — 두 텍스트 열을 읽고 다른 한 줄을 찾아 세 버튼 중 하나를 누르는 조작은 이동으로 감싸도 본질적으로 “읽고 비교하는” 문서 작업이라는 뜻으로 재진단함. 참여자가 직접 방향(“아케이드 + 메타적 요소”)을 제시했고, 이번에는 이론 조사 대신 두 구체적 후보를 스케치해 제시함: (A) **SIGNAL STRIKE** — 두더지잡기/과일 베기형, 팝업된 신호가 진짜/가짜인지 즉시 판단해 반응하는 방식, (B) **CORE DEFENSE** — 미사일 커맨드형, 화면 가장자리에서 다가오는 가짜 신호를 요격하는 방식. 이동·충돌 로직이 필요 없어 마감을 앞둔 시점의 구현 리스크가 더 낮다는 점을 근거로 (A)를 추천했고, 참여자가 이를 선택함.

핵심 설계 결정: 판정 자체(읽기·대조·분류)를 완전히 폐기하고 조작의 본질을 반응 속도로 바꾼다.

- **텍스트 대조를 색·기호 즉시 판별로 교체**. **MORI**의 기록 / 이 세션의 주장 2~3줄 비교 카드를 없애고, 신호 하나당 **rose** × (가짜) 또는 **cyan** ◆ (진짜) 아이콘 하나만 보여줌. 판단에 걸리는 시간을 “읽는 시간”에서 “보는 즉시 아는 시간”으로 줄이는 것이 이번 재설계의 유일한 목적임.
- **웨이브(6판) 구조를 폐기하고 60초 단일 런 + 시간 기반 escalation으로 전환**. 신호 등장 간격(900ms→340ms), 지속시간 (1.45초→0.7초), 동시 개수(1→4)가 경과 시간만으로 계산되는 순수 함수 (**getSpawnIntervalMs** / **getSignalLifespanMs** / **getMaxConcurrentSignals**)로 구현되어, 6웨이브 대신 한 판 안에서 연속적으로 어려워짐.
- **이동+도킹 레이어에서 이미 검증한 패턴(고정 슬롯, conic-gradient 링, 클릭+번호 키 이중 입력)을 그대로 재사용하고 “걸어가기”만 제거**. 터미널 5개를 신호 슬롯 6개(2행 3열)로 바꾸고, 각 슬롯에 항상 번호 키(1~6)를 붙여 요원 이동 없이 즉시 반응하게 함.

- **비대칭 페널티 원칙은 그대로 유지하되 대상만 바꿈.** “위조 승인이 오인보다 무겁다”는 원칙을, “진짜를 잘못 클릭하는 것(코어 무결성 -1)이 가짜를 놓치는 것(콤보만 끊김, 무결성 무변화)보다 무겁다”로 재적용. 새 함수 `getWrongClickLoss(deepVerify)` 만 추가하고 원칙 자체는 재사용.
- **ARCHIVE LENS / DEEP VERIFY / MORI REQUEST /기억 조각 시스템은 트리거만 재정의하고 계산 로직은 재사용.** ARCHIVE LENS 는 “판정 공개”에서 “4초 동안 신호 지속시간 1.6배” 슬로모션으로, DEEP VERIFY 는 “웨이브당 1회 다음 판정”에서 “런당 1회 5초 2배 윈도”로 바뀌었지만, 충전 수 계산(`getArchiveLensCharges`)과 기억 조각 보상(`getFragmentReward` , `getMoriArchiveRecord`)은 한 줄도 바꾸지 않음.

반영 내용:

- `game-logic.js` 를 웨이브/세션/사실-카탈로그 관련 함수(`getWaveConfig` , `createSessionClaims` , `createFactCatalog` , `getWrongJudgmentLoss` , `getAuditGateStatus` , `getSyncRecoveryIndex` 등)를 전부 제거하고, 60초 escalation 곡선과 반응형 채점 함수(`getSpawnIntervalMs` , `getSignalLifespanMs` , `getMaxConcurrentSignals` , `pickSignalKind` , `scorePurge` , `getWrongClickLoss`)로 다시 작성. `clamp` / `seedFromString` / `createRandom` / `formatScore` 와 기억 조각·아카이브 레코드 함수(`awardMemoryFragment` , `getArchiveLensCharges` , `getFragmentReward` , `getMoriArchiveRecord`)는 변경 없이 재사용
- `main.js` 의 세션 흐름 전체(터미널 상태머신, 아바타 이동, 도킹/언도킹, 룸 렌더링)를 제거하고, 슬롯 상태머신(`idle` → `fake|genuine` → `hit|wrong|missed` → `idle`)과 단일 `requestAnimationFrame` 루프(`gameLoop`)로 재작성. 클릭과 번호 키(1~6)가 같은 `activateSlot(index)` 를 호출해 입력 경로를 통일
- 탭 이탈 시 60초 타이머와 모든 슬롯의 남은시간을 함께 멈추고, 복귀 시 멈춘 시간만큼 타임스탬프를 그대로 밀어 넣는 방식으로 `handleVisibilityChange` 를 재작성 — 이전에는 탭 이탈이 “새 세션 재추론” 트리거였지만, 이제는 판정 대상 자체가 사라져 재추론할 것이 없으므로 단순 일시정지로 단순화
- `index.html` 의 터미널 룸·세션 카드·트리아지 3버튼·라운드 임팩트 오버레이를 신호 슬롯 6개 + HUD로 교체하고, `styles.css` 에서 대응하는 죽은 규칙(`.terminal-room` , `.terminal-node` , `.triage-*` , `.session-card` , `.claim-*` , `.round-impact*` , `.audit-progress*`)을 전부 제거 — 빌드 후 CSS 번들이 48.5KB→37.7KB로 줄어든 것으로 삭제 범위를 재확인
- Puppeteer로 재검증: 가짜 신호를 클릭하면 정화·점수·콤보가 오르는지, 진짜 신호를 클릭하면 코어 무결성이 줄고 콤보가 끊기는지, 오클릭 3회로 무결성이 0이 되면 결과 화면에 `NULL` 랭크로 도달하는지, F (ARCHIVE LENS)·D (DEEP VERIFY) 키가 실제로 상태를 바꾸는지, DEEP VERIFY 활성화 중 정화 점수가 정확히 2배로 채점되는지, 번호 키 1~6이 클릭과 동일하게 동작하는지 확인 — 콘솔 에러 0건

로그라이트 강화 선택 추가: 순발력 외의 의미 부여

이게임,,, 재밌냐?단순히 순발력 말고는 무슨의미가있는지 모르겠네

음,, 좀더 게임적인 요소나 메타적인 소로다도 추가해봐. ㄱㄱㄱㄱ

SIGNAL STRIKE로 재설계한 뒤 참여자가 직접 플레이해보고, 게임 형태 자체에는 만족했지만 “반응 속도 말고 실제로 뭘 결정하는지 모르겠다”는 새로운 지적을 함. 스스로 코드를 다시 짚어 진단: ARCHIVE LENS / DEEP VERIFY 를 언제 쓸지, MORI REQUEST 가 이번 런에 뭘 요구하는지는 선택이었지만, 매 순간의 핵심 동작(신호 하나를 보고 처리) 자체에는 트레이드오프가 전혀 없어 “본 대로 반응하는” 것 외에 결정할 지점이 없었음. “게임적인 요소나 메타적인 요소를 추가해보자”는 요청에 대해, 마감을 감안해 완전한 7번째 장르 교체 대신 기존 반응형 루프를 유지한 채 그 위에 짧은 빌드 선택을 엮는 절충안을 제시함.

핵심 설계 결정: 강화는 전부 순수 상승이 아니라 트레이드오프로 설계해, “무엇을 고를지”가 실제 결정이 되게 한다.

- 6종 강화(콤보 특화/안전한 손/여유 확보/고위험 배팅/렌즈 숙련/코어 보강)를 모두 득과 실이 함께 있는 쌍으로 정의(`BUFF_DEFINITIONS`). 예: 콤보 특화는 콤보 보너스 +50%지만 오클릭 시 무결성 손실이 추가로 1칸 더 붙음
- 60초 런 중 15초·30초·45초 지점마다 2장을 제시해 1장을 고르게 하고, 제시된 2장은 고르든 안 고르든 풀에서 함께 제외해 한 런에 정확히 3종(6종 중)만 겹치지 않게 갖도록 설계 — 매 런 다른 조합이 되어야 “이번 런은 콤보형으로 가야겠다”처럼 방향을 결정하는 의미가 생김

- `scorePurge / getWrongClickLoss` 에 하위호환 기본값을 유지한 채 선택적 `modifiers / bonusLoss` 매개변수를 추가해 강화 효과를 적용 — 기존 호출부와 테스트는 전부 그대로 통과함
- 강화 선택 중에는 게임을 완전히 멈추고, 고른 뒤에는 멈춘 시간만큼 모든 신호의 마감시각과 다음 스폰 시각을 그대로 밀어 넣어 60초 총 플레이 시간이 그대로 보존되게 함 — Page Visibility 일시정지에서 쓰던 “타임스탬프 밀어 넣기” 트릭을 그대로 재사용, 새 일시정지 로직을 만들지 않음
- Puppeteer로 검증: 강화 픽이 정확히 15/30/45초 경과 시점에 열리는지, 3번의 픽이 서로 겹치지 않는 6종을 정확히 소진하는지, 번호 키(1/2)와 클릭 모두로 고를 수 있는지, 두 번의 정지-재개를 거쳐도 60초 런이 정확한 시점에 끝나 결과 화면에 도달하는지 확인 — 콘솔 에러 0건

손맛(juice) 보강: “정말 게임처럼 느껴지게”

좀더 발전시켜보자, 정말 게임처럼 느껴지게

너가 만족할때까지, 비판적으로 체크해가며 작업해보자

강화 선택으로 순발력 외의 결정을 넣은 뒤, 참여자가 “정말 게임처럼 느껴지게” 더 발전시켜 달라고 요청함. AI 스스로 만족할 때까지 비판적으로 점검하라는 요청을 받아, 방어적으로 “이미 게임처럼 느껴진다”고 답하지 않고 부족한 지점을 직접 진단함: 코어 무결성이 HUD의 숫자·팝(pip)으로만 존재해 “무엇을 지키고 있는지”가 화면에 없었고, 정화·오클릭 모두 슬롯 하나가 반짝이는 것 이상의 반응이 없었으며, 결과 화면은 텍스트가 그냥 나타날 뿐이었음. 이 세 지점을 메커닉은 건드리지 않고 순수 시각·청각 피드백 (Steve Swink, 『Game Feel』 — 이 프로젝트가 SIGNAL TRIAGE 재설계 때부터 이미 근거로 삼던 이론)으로만 보강함.

반영 내용:

- signal-field 정중앙(원래 비어 있던 자리)에 `core-orb` 를 추가 — 6개 슬롯이 실제로 감싸고 지키는 대상이 눈에 보이는 오브젝트가 되도록 함. 무결성 3/2/1에 따라 cyan→amber→rose로 색이 바뀌고 호흡하듯 계속 펄스하며, 정화 시 짧게 커졌다가(emerald) 오클릭 시 흔들리며(rose) 반응하고, 무결성이 0이 되면 확대되며 사라지는 `shatter` 애니메이션 후 결과 화면으로 전환됨
- 오클릭 시 `signal-field` 전체에 짧은 화면 흔들림(`shakeSignalField`)을 추가 — 기존 슬롯 자체의 rose 흔들림에 더해 실수의 무게를 화면 전체로 확장
- 콤보 4/8/12 달성 시 코어 위로 “COMBO ×N!” 텍스트가 튀어 오르는 콜아웃과, 이전에 정의됐지만 어디서도 호출되지 않던 `audio.core()` 를 되살려 마일스톤 사운드로 사용 — 새 사운드를 만들지 않고 죽은 코드를 재활용
- 결과 화면 점수를 즉시 표시하지 않고 0에서 최종 점수까지 900ms 동안 ease-out으로 세는 카운트업(`animateScoreCountUp`)과, 랭크 글리프가 확대·회전하며 등장하는 스탬프 애니메이션을 추가. S랭크는 amber 금빛, NULL랭크는 rose로 색이 갈려 결과를 한눈에 체감하게 함
- 모든 신규 애니메이션에 `prefers-reduced-motion` 분기를 추가해, 이 환경에서는 점수가 즉시 최종값으로 표시되고 코어·흔들림·콜아웃 애니메이션이 꺼지되 색상·상태 변화(무결성 색, 랭크 색)는 동일하게 유지됨
- Puppeteer로 검증: 콤보 4에서 콜아웃이 뜨고 코어가 pulse 상태로 전환되는지, 오클릭 시 코어와 무결성 팝이 함께 amber/rose로 바뀌는지, 무결성이 0이 될 때까지 반복해도 정상적으로 결과 화면에 도달하는지, 결과 화면 점수가 시간에 따라 실제로 증가하다 최종값에 도달하는지, NULL / S 랭크에 따라 `resultGlyph` 의 `data-rank` 가 정확히 반영되는지 확인 — 콘솔 에러 0건

강화 풀 다양화 + 잠금 해제형 메타 진행

더 밀어붙이자. 각각의 대상들은,, 네가 웹 검색을 통해서 보충할 수 있나?

손맛 보강 이후에도 남은 세 가지 얇은 지점(오디오·강화 다양성·런 간 반복 동기)을 직접 짚어 알렸고, 참여자가 “더 밀어붙이자”며 웹 검색으로 실제 레퍼런스를 찾아 보강할 수 있는지 물음. 웹 검색으로 로그라이트 설계 아티클을 확인한 결과 “variety within consistency” — 핵심 루프는 고정하고 빌드·조합은 매번 달라야 한다는 원칙과, “achievement가 죽음을 덜 아프게 만든다”는 메타 진행 관찰을 근거로 다음을 반영함:

- 강화를 6종에서 10종으로 확장(역풍, 냉정 유지, 신호 왜곡, 속사 모드 추가). 기존에는 매 런마다 같은 6종을 순서만 바꿔 제시했는데, 이제 `RUN_BUFF_POOL_SIZE` (=6)만큼을 10종 중에서 무작위로 뽑아 이번 런의 메뉴 자체가 매번 달라지도록 바꿈
- 신호 왜곡·속사 모드 2종은 `unlock.minFragments: 2` 로 기억 조각 2개 이상을 모아야 강화 풀에 들어옴 — 처음 몇 판은 안전한 기본 강화만 보이고, 반복 플레이로 기억 조각을 모을수록 더 높은 변동성의 강화가 열림. 인트로 화면에 “기억 조각을 N개 더 모으면 고위험 강화 2종이 열립니다” 안내를 추가해 눈에 보이는 다음 목표로 삼음
- `pickSignalKind` 에 `genuineChance` 매개변수를 추가(기본값은 기존 `GENUINE_CHANCE` 그대로 유지하는 하위호환 확장)하고, 신규 필드 `genuineChanceBonus / spawnIntervalScale` 을 `runtime.buffs` 에 연동해 신호 왜곡 (진짜 비율 상승)과 속사 모드 (등장 간격 단축)를 구현
- `tests/game-logic.test.js` 에 강화 풀이 `RUN_BUFF_POOL_SIZE` 이상을 갖는지, 잠긴 강화가 기억 조각 0개에는 절대 섞이지 않고 최대치에는 반드시 포함되는지 검증하는 테스트를 추가(총 17개 통과)

배경 오디오: 합성 전용에서 CCO 외부 음원 허용으로

같은 요청에 이어 참여자가 Freesound API 자격증명을 전달하며 실제 외부 음원 사용을 지시함(\$5.1에 기술 세부 사항 기록). 이전까지 이 문서와 README는 “외부 음원을 사용하지 않는다”를 원칙처럼 서술했는데, 참여자가 “다른 가능성을 닫는 문서 내용을 전부 없애라”고 명시적으로 요청해 그 서술을 모두 “필요하면 출처·라이선스를 표에 남기고 추가할 수 있다”는 열린 서술로 고쳤음(\$5, README 참고). 기존 합성 SFX(정화·오클릭·경보·콤보·결과)는 그대로 두고, 배경 앰비언트 루프 1개만 CCO 외부 음원으로 교체해 오디오에 실제 질감을 더함.

MORI 대화가 실시간으로 “말하듯” 나오도록

UI, UX 및 캐릭터성도 좀더 살펴보자, 예를들면 튜토리얼에서 모리가 말하는 것처럼 이야기하한다거나,, 하는식으로

참여자가 UI/UX와 캐릭터성을 더 살려 달라며, 튜토리얼에서 MORI가 실제로 말하는 것처럼 느껴지게 해 달라고 예를 들어 요청함. 코드를 살펴보니 두 가지 문제가 겹쳐 있었음: (1) `src/game-logic.js` 에 내보내져 있던 `PLAY_INSTRUCTION` 이 어디서도 호출되지 않는 죽은 코드였고, (2) 첫 화면 문구(`intro-heading / intro-message`)가 “진짜인지 증명하세요”, “게임 쿠키가 삭제되었습니다”처럼 MORI 1인칭이 아니라 시스템 안내문 톤으로 쓰여 있었음. 두 문제를 한 번에 고침:

- 첫 방문·재방문·아카이브 완료·기억 초기화 4개 분기의 인트로 헤딩·본문을 전부 MORI의 1인칭 반말 화법으로 다시 씀(예: “진짜인지 증명하세요” → “코어를 지켜줘. 그럼 내 첫 기억 조각을 돌려줄게”). 기존 `MORI_STATES` 의 플레이 중 대사 톤(짧고 담담한 반말)과 통일함
- 죽어 있던 `PLAY_INSTRUCTION` 을 실제로 연결함 — `startGame` 에서 `round-heading / board-instruction` 을 이 상수의 `prompt / instruction` 값으로 채워, 지금까지 HTML에 손으로 중복 적어 두던 문구를 단일 출처로 정리함
- `typewriteText(el, text, { speedMsPerChar, onDone })` 를 추가해 MORI의 모든 대사(인트로 헤딩→본문 순차 타이핑, 플레이 중 상태 대사, 결과 화면 대사)가 즉시 나타나지 않고 한 글자씩 나타나도록 바꿈. 타이핑 중에는 깜빡이는 커서 (|)를 붙여 “지금 말하고 있다”는 신호를 줌
- 인트로 화면은 헤딩이 먼저 다 타이핑된 뒤에야 본문이 시작되도록 체이닝(`speakIntroLines`)해, 짧은 2줄 모놀로그처럼 읽히게 함. 다만 정책 선택 버튼은 타이핑 중에도 항상 클릭 가능하게 남겨 뒀, 심사자가 빠르게 플레이를 시작하는 데는 지장이 없음
- 결과 화면에서 스타일 remark(예: “손이 브라우저의 반응속도보다 빨랐습니다”)를 대사 뒤에 갖다 붙이던 방식은 타이핑 중인 문자열을 인터벌이 매 프레임 원본 텍스트로 덮어써 remark가 사라지는 경쟁 상태를 만들 수 있었음 — remark를 먼저 문자열로 합친 뒤 그 완성된 문장을 한 번에 타이핑하는 방식으로 바꿔 근본 원인을 제거함
- `prefers-reduced-motion` 환경에서는 타이핑 애니메이션과 커서를 모두 건너뛰고 최종 텍스트를 즉시 표시(이전 애니메이션들과 동일한 규칙)
- Puppeteer로 검증: 인트로 진입 후 300ms 시점에 본문이 실제로 부분 문자열 상태인지, 타이핑 도중에도 정책 선택 버튼이 `disabled` 가 아닌지, 타이핑이 끝난 뒤 최종 문장이 원래 의도한 전체 텍스트와 일치하는지 확인 — 콘솔 에러 0건

아이템(강화) 무제한 확장: “뱀서류처럼, 근데 개수 제한 없이”

아이템(업그레이드)를 좀더 다방면으로 만들어버리는건 어때...? 뱀서류 처럼 하는거지.

시간은 신경쓰지마, ai agents를 통해서 충분히 작업 가능해, 얼마든지.

특정상황마다 모리가 대사도 치고, 뱀서류랑 다른건 아이템 (혹은업그레이드)_개수에 제한은아예없는걸 특징으로 삼자. 아이템(업그레이드)를 엄청 많이 만들어야겠지.

강화 시스템이 이미 있었는데도, 참여자가 “더 다방면으로, 뱀서류(Vampire Survivors류)처럼”라는 방향을 제안함. 다만 60초 단일 런에 뱀서라이크의 수십 종 무기·시너지·수십 분 축적 리듬을 그대로 옮기는 건 이 게임 구조와 맞지 않는다고 판단해, 먼저 절충안을 제시하고 확인받음(강화 수 확장/스택 도입 vs 그대로 유지) — 참여자가 “시간 신경 쓰지 말고 에이전트로 얼마든지”라고 답하며 스케일을 키우는 쪽으로 확정. 뒤이어 “뱀서류와 다른 건 개수 제한이 아예 없는 것”이라는 명시적 차별점과 “MORI가 상황마다 대사를 친다”는 요구가 추가됨.

설계 결정: 15/30/45초 3회·6종 소진형이던 강화 픽을 7초부터 6.5초 간격으로 반복되는 픽(60초 런에 약 8회)으로 바꾸고, 같은 강화를 몇 번이든 다시 뽑아 무제한으로 스택할 수 있게 함(`getBuffPickTriggers`). 뱀서라이크와의 차이점을 “슬롯 제한 없음, 교체 없음”으로 명시적으로 삼음.

콘텐츠 생성: 병렬 서버에이전트 3개(공격/점수형, 방어/자원형, 와일드카드/템포형, 각 10종)에 허용된 effects 필드 목록과 “모든 강화는 업사이드와 다운사이드를 반드시 함께 가진다”는 기존 원칙을 프롬프트에 명시해 초안을 받고, 참여자를 대신해 직접 검수함 — 필드 오남용(존재하지 않는 키 사용), 업사이드만 있는 항목, 밸런스가 과한 조합을 확인해 전부 통과시킴(불합격 항목 없음, 문구만 일부 다듬음). 여기에 새 메커닉 6종(보호막·부활·연쇄 반응·마일스톤 고정 보너스·DEEP VERIFY 추가 사용권 2종)은 런타임 로직이 필요해 참여자가 직접 설계·구현함. 최종 강화 풀은 10(기존)+6(신규 메커닉)+30(에이전트 초안) = 46종.

새 메커닉 5종(순수 숫자 재조합이 아닌 새 동작): - `shieldCharges` : 다음 오클릭을 완전히 무효화(무결성 손실 없음, 콤보 유지, `wrongClicks` 통계에도 반영 안 됨 — 온전히 없었던 일로 처리) - `reviveCharges` : 코어 파괴 시 1회 무결성 1칸으로 부활 - `chainClearChance` : 가짜 정화 시 확률적으로 다른 가짜 하나를 자동 정화(스노우볼) - `milestoneScoreBonus` : 콤보 4/8/12 달성마다 고정 점수 보너스 - `extraDeepVerifyUse` : 기본 런당 1회이던 DEEP VERIFY 사용 횟수를 늘림

무제한 스택의 안전장치: 상한 없는 반복 픽이 게임을 깨뜨리지 않도록, 강화 개수를 제한하는 대신 매 계산식에 하한선을 둬 — `scorePurge` 의 `comboScale / scoreScale` 유효 배율은 각각 0과 0.15 이하로 못 내려가고, 실행 지속시간은 220ms, 스펀 간격은 90ms 밑으로 못 내려가며, DEEP VERIFY 창은 1초 밑으로 못 줄어듦. 즉 디버프를 아무리 많이 쌓아도 “매우 나빠짐”까지만 가고 수학적으로 뒤집히거나 0으로 나누는 상태는 나오지 않음.

UI: 강화 텍스트 요약에 스택 횟수를 표시(콤보 오버드라이브 $\times 2$)하고, 코어 주위로 보유한 강화마다 점 하나씩이 천천히 도는 `#buff-orbit` 링을 추가해 “지금 얼마나 쌓였는지”를 텍스트를 읽지 않아도 한눈에 보이게 함. 보호막·부활 보유량은 별도 `#resource-strip` 에 표시.

MORI 대사: 46종 강화 전부에 `moriLine` (1인칭 반말 반응 한 줄)을 채워, 강화를 고를 때마다 `pulseMoriState` 로 그 강화에 맞는 짧은 반응이 뜨도록 연결 — “상황마다 대사를 친다”는 요구를 강화 46종 전체에 적용함.

검증 중 발견한 버그: Puppeteer로 보호막을 직접 고른 뒤 진짜 신호를 클릭해 소모되는지 확인하는 과정에서, `runtime.shieldCharges` 는 실제로 정확히 소모되고 있었지만 화면의 `#resource-strip` (보유 자원 표시)이 픽 시점 이후로는 다시 그려지지 않아 소모가 화면에 반영되지 않는 버그를 발견함. 원인은 `updateActiveBuffsReadout()` 가 강화를 고른 순간에만 호출되고, 실제 소모가 일어나는 `activateSlot` 경로에서는 호출되지 않았던 것 — `updateHud()` (모든 정화·오클릭마다 호출됨) 안에서 함께 호출하도록 한 줄 추가해 고침. 이 버그는 게임 로직 자체가 아니라 화면 갱신 누락이었고, Puppeteer로 “고른 직후”와 “소모 직후”를 둘 다 스냅샷 비교하지 않았다면 통과했을 문제라, 그냥 “동작한다”로 넘기지 않고 전/후 상태를 직접 대조한 것이 실제로 버그를 잡아낸 경우임.

검증: 60초 런 전체를 자동 플레이해 실제로 약 8회의 강화 픽이 반복 간격대로 열리는지, 같은 강화를 두 번 골랐을 때 텍스트 요약과 오빗 링 배지에 $\times 2$ 가 정확히 반영되는지 확인(실측: 8회 픽, 콤보 오버드라이브 $\times 2$ 포함 7개 항목, 오빗 점 7개·배지 1개로 정확히 일치, 최종 점수 55,007·S랭크). 별도로 보호막을 골라 실제 오클릭을 막는지(무결성·오클릭 수 불변, 보유량 정확히 감소), DEEP VERIFY 추가 사용권을 골라 런당 두 번째 발동이 실제로 가능한지(`disabled` 해제, 두 번째 발동 후에도 `active` 상태 전환)를 각각 실제 브라우저에서 재확인 — 두 시나리오 모두 콘솔 에러 0건.

3. AI가 지원한 구현 영역

기획

- HTTP의 무상태성과 쿠키의 상태성을 “MORI가 위조 세션을 가려내는 기술적 근거”로 설정
- 실제 브라우저 텔레메트리 14개 사실을 2~3개씩 결합한 세션 주장으로 심사하는 6웨이브 게임 루프 설계
- 재방문, 탭 이탈, 복수 탭 감지를 메타 서사로 연결
- 기억 조각 6개를 쿠키에 저장하는 중기 진행 목표 설계
- 판정 로직은 그대로 두고 “세션에 도달하는 방식”만 교체해 개발 리스크를 낮추는 절충 설계 — 룸에 배치된 여러 터미널이 동시에 켜지고 요원을 직접 이동시켜야 카드가 열리는 이동+도킹 레이어와, 그로 인해 생기는 “어디부터 갈지” 실시간 우선순위 판단 설계
- (텍스트 판정 자체를 폐기하고 대체) 색·기호로 즉시 구분되는 신호를 60초 안에 반응 처리하는 순수 반응형 아케이드 설계 — 고정 슬롯·번호 키 입력·비대칭 페널티 원칙은 이전 재설계에서 재사용하고, 웨이브 구조와 텍스트 대조만 폐기

개발

- Vite 기반 정적 웹 프로젝트 구성
- 키보드 숫자 구역 지정과 마우스 클릭 판정을 동일한 판정 모델로 통합 구현(칩 선택 개념 없이 한 번에 하나의 세션만 제시)
- 웨이브별 목표 세션 수·결합 사실 개수·세션당 제한시간, 연속 정답 보너스, 무결성, 랭크 판정 구현
- 조각 수에 따라 성장하는 제한 자원 ARCHIVE LENS (웨이브당 1회, 지금 세션의 실제 판정 공개), 키보드 F 입력 구현
- VERIFIED 성공에만 기억 조각을 지급하고 조각마다 MORI 서사 파일을 공개하는 보상 루프 구현
- 연속 정답 SYNC 를 HUD와 MORI 반응 상태에 연결
- 3연속 정답 시 가장 최근 오류를 5 상태로 바꾸고 무결성을 회복하는 1회성 SYNC RECOVERY , 복구 전용 MORI 컷인과 A랭크 상한 구현
- 웨이브당 1회, 다음 판정 한 번에 정답 추가 점수와 오답 2칸 손실이 적용되는 DEEP VERIFY 토글, 저무결성 잠금, MORI 상태·대사·판정 컷인 구현
- 마지막 웨이브에서 DEEP VERIFY 보너스를 +700점으로 올리고 CORE EXPOSED MORI 상태와 코어 색상 연출을 연결하는 최종 웨이브 연출 구현
- 런별 도전의 순환·진행·단일 보너스 지급, 플레이 상태줄, 완료 컷인과 캐릭터 대사, 결과 판정을 연결하는 MORI REQUEST 구현
- VERIFIED GATE 에 현재 웨이브·성공·실패·보상까지 이번 웨이브 진행 상황을 표시하고 더 이상 성공할 수 없는 런을 즉시 구분
- 웨이브 종료 시점 판정에서 경고색, Web Audio 신호, MORI 반응 상태를 동시에 전환
- Cookie Store 비동기 저장과 document.cookie 폴백 구현
- 세션 카운트다운의 진행률을 단일 CSS 커스텀 프로퍼티(--session-progress)로 표현해 카드 아래 막대가 줄어들도록 구현 — 세션 카드가 원래도 정적 레이아웃이라 prefers-reduced-motion 전용 분기 불필요
- 외부 파일 없는 Web Audio 효과음 구현
- GitHub Pages Actions 워크플로 작성
- (이후 터미널 이동+도킹 재설계로 대체됨) 세션 제시·카운트다운을 담당하던 requestAnimationFrame 루프 (updateSessionFrame)와 순수 판정 함수(judgeSession , expireSession , advanceSessionOrConclude)로 심문관형 판단 메커닉 구현. 씬 전환마다 다르던 상단 지시문을 웨이브가 바뀌어도 동일한 공통 상수(PLAY_INSTRUCTION)로 유지 — 이 원칙은 현재 구조에서도 그대로 유지됨
- (SIGNAL STRIKE 재설계로 대체됨) 요원 이동·터미널 도킹·근접 판정을 담당하던 단일 requestAnimationFrame 루프 roomLoop 와 judgeSession / handleTerminalTimeout / settleTerminal / checkWaveCompletion / concludeWave 로 이동형 심문관 판단 메커닉 구현
- 플레이 통계(위험 검증 승리 횟수·무결점 여부·LENS 사용 횟수)로 4가지 런 스타일 태그를 계산하는 순수 함수 getRunStyleTag / getRunStyleRemark 구현, 결과 화면 MORI 대사에 반영 — SIGNAL STRIKE에서도 입력값(콤보·오클릭·DEEP VERIFY 정확 수)만 바꿔 재사용

- 위조 세션 승인(무결성 -2)과 진짜 세션 오인 거부(무결성 -1)를 구분하던 `getWrongJudgmentLoss(mistakeType, deepVerify)` 를, “진짜 오클릭”이라는 단일 실수 유형에 대한 `getWrongClickLoss(deepVerify)` 로 단순화해 재구현 (기본 -1, DEEP VERIFY 중 -2 원칙은 그대로)
- 60초 escalation 곡선(`getSpawnIntervalMs / getSignalLifespanMs / getMaxConcurrentSignals`), 슬롯 상태머신, 단일 게임 루프 `gameLoop` 로 SIGNAL STRIKE 반응형 판정 메커니즘 구현 (§4 “SIGNAL STRIKE 반응형 판정” 참고)
- ARCHIVE LENS 를 4초 슬로모션 버퍼(신호 지속시간 1.6배), DEEP VERIFY 를 5초 2배 채점 원도로 재구현. 두 자원 모두 충전 계산·잠금 로직은 이전 재설계에서 그대로 재사용
- Page Visibility 일시정지 로직을 “새 세션 재추론”에서 “타임스탬프를 멈춘 시간만큼 그대로 밀어 넣는” 방식으로 단순화 — 판정 대상(세션 주장)이 사라지면서 재추론할 대상 자체가 없어졌기 때문
- 60초 런 중 15/30/45초 지점마다 게임을 멈추고 트레이드오프가 있는 강화 카드 2장 중 1장을 고르게 하는 로그라이트 픽 시스템(`openBuffPick / chooseBuff / closeBuffPick`) 구현. 6종 강화(`BUFF_DEFINITIONS`)를 3번에 나눠 겹치지 않게 소진해 런마다 다른 조합이 되도록 설계 (§4 “로그라이트 강화 선택” 참고)
- `scorePurge / getWrongClickLoss` 에 하위호환 기본값이 있는 선택적 `modifiers / bonusLoss` 매개변수를 추가해 강화 효과(콤보·점수 배율, 오클릭 손실 가감)를 반영. 기존 호출부·테스트는 변경 없이 그대로 통과

검증

- 6개 웨이브 인덱스 전부에서 목표 세션 수·결합 사실 개수·세션당 제한시간·판독 불가 비율이 배열 길이와 단조 방향을 지키는지(`getWaveConfig`) 테스트
- 동일 시드에서 `createSessionClaims` 가 동일한 세션 구성을 재현하는지, 각 세션이 결합 사실 개수만큼 서로 다른 사실을 포함하는지, 진짜 세션은 거짓/오염 줄이 0개, 위조 세션은 거짓 줄이 정확히 1개, 판독불가 세션은 오염 줄이 정확히 1개인지 테스트
- 대량 샘플링으로 판독 불가(`corrupted`) 세션 비율이 웨이브가 진행될수록 실제로 늘어나는지 테스트
- 위조 승인·진짜 오인·판독불가 오판정 세 가지 실수 유형과 DEEP VERIFY 여부 조합에서 `getWrongJudgmentLoss` 가 정확한 무결성 손실을 반환하는지 테스트
- 점수·난이도·결말 경계값 테스트
- SYNC RECOVERY 가 3연속 이전에는 발동하지 않고 가장 최근 오류를 런당 한 번만 복구하며, 복구 런이 S랭크를 받지 않는지 테스트
- DEEP VERIFY 의 기본 보너스 350점·마지막 웨이브 보너스 700점과 실패 시 무결성 2칸 손실 경계값 테스트
- MORI REQUEST 3종 순환과 완료 경계, +600점 단일 지급, REQUEST SEALED 상태와 결과 판정 테스트
- `getRunStyleTag` 가 RECOVERY / RISK / PRECISION / SPEED 네 조건과 빈 통계 기본값을 각각 정확히 반환하는지 테스트
- 기존 버전 쿠키가 기억 조각 0개인 버전 2로 안전하게 이전되는지 테스트
- 쿠키 데이터 직렬화 시 허용 필드만 보존되는지 테스트
- 헤드리스 브라우저로 마우스 클릭·키보드 숫자 키 두 경로 모두 세션 판정이 정상 동작하는지, 웨이브 중 오답을 내도 무결성이 남아 있으면 웨이브가 계속되고 목표 세션 수를 모두 처리해야 종료되는지 점검
- 가상 시계 하네스로 항상 VERIFIED 만 판정했을 때 정답(손실 0)·위조 승인(손실 2)·진짜 오인(손실 1) 세 종류의 무결성 변화가 실제로 다르게 나타나는지 직접 재현해 비대칭 페널티를 확인
- ARCHIVE LENS 가 지금 세션의 실제 판정을 `data-reveal-zone` 으로 정확히 표시하는지, DEEP VERIFY 토글이 다음 판정 한 번에만 적용되고(`dataset.active`) 소진 후 자동으로 꺼지는지 점검
- 실제 브라우저 상태(`matchMedia.navigator.onLine`·innerWidth·쿠키)로부터 각 사실의 truth/lie/decoy 문구를 독립적으로 재구성해 세션 주장의 각 줄을 판정하는 자동 플레이가, 소스 코드 지식 없이 오직 화면에 보이는 정보만으로 6웨이브 전체를 무결성 손실 없이 클리어하고 S랭크·6/6으로 결과 화면에 도달하는지 확인(재구성한 36개 세션 전부에서 판정 불가 줄 0건) — 콘솔 에러 0건
- (터미널 이동+도킹 재설계 이후, SIGNAL STRIKE 이전 상태) Puppeteer로 로컬 Chrome을 직접 조작해: 클릭 이동이 목표 터미널에 정확히 도킹하고 세션 카드가 열리는지, 판정 후 대기열의 다음 세션이 같은 슬롯에 즉시 이어지는지, 무결성이 0이 되면 결과 화면에 NULL 랭크로 도달하는지, 터미널에 도킹하지 않고 카운트다운만 흘러보내면 무결성 손실 없이 “놓침”만 기록되는지, 방향키 입력만으로 아바타 좌표가 실제로 변화하는지 확인 — 다섯 시나리오 모두 콘솔 에러 0건

- (SIGNAL STRIKE) Puppeteer로 재검증: 가짜 신호 클릭 시 정화·점수·콤보 상승, 진짜 신호 클릭 시 코어 무결성 하락과 콤보 초기화, 오클릭 3회로 무결성이 0이 되면 결과 화면에 NULL 랭크로 도달, F / D 키로 ARCHIVE LENS · DEEP VERIFY 가 실제로 활성화되고 DEEP VERIFY 중 정화 점수가 정확히 2배로 채점되는지, 번호 키 1~6이 클릭과 동일하게 슬롯을 정화하는지 확인 — 콘솔 에러 0건
- `scorePurge(1, 8, 1, {})` 가 수정 전과 동일한 값을 내는지(하위호환), `comboScale / scoreScale` 을 각각 올리고 내렸을 때 점수가 예상대로 오르내리는지, `getWrongClickLoss` 에 `bonusLoss` 를 더하거나 빼도 0 미만으로는 내려가지 않는지 테스트
- `BUFF_DEFINITIONS` 가 한 런에 필요한 개수(`RUN_BUFF_POOL_SIZE` = 3번 픽 × 2장 = 6) 이상이고 id가 전부 고유하며 각 항목에 이름·설명·effects가 채워져 있는지, 잠긴 강화가 기억 조각 0개에는 섞이지 않고 최대치에는 반드시 포함되는지 테스트
- Puppeteer로 실제 브라우저 조작: 강화 픽이 정확히 15,000ms/30,000ms/45,000ms 경과 시점에 열리는지, 3번의 픽에서 제시되는 카드 쌍이 겹치지 않고 6종을 정확히 소진하는지, 번호 키(1/2)로 고르면 오버레이가 닫히고 상태줄에 강화 이름이 누적되는지, 두 번의 정지-재개 이후에도 60초 런이 정확한 시점에 끝나 결과 화면에 도달하는지 확인 — 콘솔 에러 0건

4. 핵심 기술 구조

SIGNAL STRIKE 반응형 판정

신호는 더 이상 텍스트 주장으로 제시되지 않습니다. 고정된 6개 슬롯 중 idle 상태인 슬롯에 `pickSignalKind()` 가 45% 확률로 `genuine`, 55% 확률로 `fake` 를 뽑아 배정하고, 슬롯은 `idle → fake|genuine → hit|wrong|missed → idle` 을 순환합니다. 진짜/가짜 구분은 슬롯에 표시되는 아이콘(× / ♦)과 색(`rose/cyan`) 하나로 끝나, 텍스트를 읽을 필요가 없습니다.

난이도는 경과 시간(`elapsedMs`)만의 함수인 세 곡선으로 결정됩니다 — 신호 등장 간격

`getSpawnIntervalMs` (900ms→340ms), 신호 지속시간 `getSignalLifespanMs` (1.45초→0.7초), 동시 허용 개수 `getMaxConcurrentSignals` (1개→4개, 최대 6). 웨이브 같은 구간 구분이 없어, 60초 동안 연속적으로 어려워집니다.

단일 `requestAnimationFrame` 루프 `gameLoop` 가 매 프레임 세 가지를 처리합니다 — 각 슬롯의 남은 시간 갱신과 만료 처리(가짜 만료는 콤보만 초기화, 진짜 만료는 조용히 idle로), 다음 신호 스폰 여부 판단(간격 도달 + 동시 개수 여유 확인), 60초 경과 시 즉시 `finishRun()` 호출. 클릭과 번호 키(1~6)는 같은 `activateSlot(index)` 를 호출해 입력 경로를 통일합니다.

로그라이트 강화 선택

`gameLoop` 는 매 프레임 `maybeTriggerBuffPick(elapsedMs)` 도 확인합니다 —

`getBuffPickTriggers(RUN_DURATION_MS)` 가 만들어 둔 목록(7,000ms부터 6,500ms 간격, 60초 런에 약 8개) 중 아직 지나지 않은 첫 값에 도달하면 `openBuffPick()` 을 호출하고 그 프레임의 나머지 처리(슬롯 만료·스폰·시간 표시)를 건너뛵니다. `openBuffPick` 은 `runtime.locked = true` 로 게임을 멈추고, 그 시점에 해금된 강화 풀(`runtime.buffPool`, `getUnlockedBuffDefinitions(fragments)` 로 계산)에서 2개를 무작위로 뽑아 오버레이에 표시합니다.

강화는 순수 함수형 데이터(`BUFF_DEFINITIONS`, 총 46종)로 정의되며, 각 항목은 `effects` 객체의 부분집합만 채웁니다. 필드는 두 갈래로 나뉩니다 — **누적형**(런 내내 유지되며 여러 번 뽑을수록 계속 더해지는 `runtime.buffs` 의 값: `comboScale`, `scoreScale`, `wrongLossBonus`, `lifespanScale`, `concurrentBonus`, `deepVerifyWindowBonusMs`, `deepVerifySpawnGenuine`, `lensDurationBonusMs`, `genuineChanceBonus`, `spawnIntervalScale`, `milestoneScoreBonus`, `chainClearChance`)와 **즉시 소비형**(뽑는 순간 한 번 적용되고 이후 실제로 소모되는 자원: `lensChargeBonus`, `healIntegrity`, `shieldCharges`, `reviveCharges`, `extraDeepVerifyUse`). 8종은 `unlock.minFragments` 로 잠겨 있어 `getUnlockedBuffDefinitions(fragments)` 가 걸러낸 목록에서만 등장합니다.

기존 로그라이트와 가장 다른 지점은 **뽑을수록 소진되는 풀이 아니라는 것**입니다. `chooseBuff(buffId)` 는 `Object.keys(runtime.buffs)` 를 도는 제네릭 루프로 누적형 필드를 전부 더하고(불리언은 OR), 즉시 소비형은 별도 분기에서 해당 자원에 더한 뒤, 어떤 필드도 풀에서 제거하지 않습니다 — 같은 강화가 다음 픽에도 다시 제시될 수 있고, 고를수록 그 효과가 계속 쌓입니다. `runtime.buffCounts / buffPickOrder` 가 몇 번씩 골랐는지·어떤 순서로 골랐는지를 추적해 상태줄 텍스트(“콤보 오버드라이브 ×2”)와 코어 주위를 도는 `#buff-orbit` 링(항목마다 점 하나, 2회 이상이면 배지 표시)에 반영합니다.

상한 없는 반복 픽은 극단적인 스택에서 수식이 깨지지 않도록 소비 지점마다 하한을 둡니다 — `scorePurge` 는 `comboScale / scoreScale` 의 유효 배율을 각각 0/0.15 밑으로 내려가지 않게 하고, `spawnRandomSignal / gameLoop` 의

스폰 간격·신호 지속시간은 각각 90ms/220ms 밑으로, `toggleDeepVerifyWager` 의 DEEP VERIFY 창은 1초 밑으로 내려가지 않습니다. 디버프를 아무리 많이 골라도 “매우 나빠짐”에서 멈추고, 수학적으로 뒤집히거나 0으로 나누지는 않습니다.

즉시 소비형 중 셋은 `activateSlot` 에서 새 분기로 소비됩니다 — 오클릭 시 `runtime.shieldCharges > 0` 이면 무결성 손실·콤보 초기화·`wrongClicks` 증가를 전부 건너뛰고 충전만 소모하고(완전히 없었던 일로 처리), 코어 무결성이 0이 되는 순간 `runtime.reviveCharges > 0` 이면 `finishRun()` 대신 무결성을 1로 되돌리고, 가짜 정화가 성공하면 `attemptChainClear` 가 `chainClearChance` 만큼의 확률로 다른 가짜 하나를 추가로 자동 정화합니다. `extraDeepVerifyUse` 는 `runtime.deepVerifyUsesLeft` (기본 1)에 더해져 `toggleDeepVerifyWager` 의 사용 가능 여부를 결정합니다.

`closeBuffPick` 은 Page Visibility 일시정지에서 쓰던 것과 같은 원리로 재개합니다 — 멈춰 있던 시간(`performance.now()` - `buffPauseStartedAt`)만큼 `runStartAt` · `nextSpawnAt` · 모든 활성 슬롯의 `deadline` 을 그대로 밀어 넣어, 강화를 고르는 데 걸린 실제 시간이 60초 런타임에서 소비되지 않게 합니다. 이 원리는 반복 픽으로 바뀐 뒤에도(픽이 3번이 아니라 8번이 되어도) 변경 없이 그대로 재사용됩니다.

정화(`activateSlot` 이 fake 슬롯에서 호출됨)는 순수 함수 `scorePurge(remainingRatio, combo, multiplier)` 로 채점됩니다 — 반응 속도(남은 시간 비율)와 콤보 수에 비례해 점수가 오르고, DEEP VERIFY 활성 중이면 `multiplier` 가 2가 됩니다. 오클릭(진짜 슬롯 클릭)은 `getWrongClickLoss(deepVerify)` 가 기본 1칸, DEEP VERIFY 중이면 2칸의 무결성 손실을 반환합니다 — “위조 승인이 오인보다 무겁다”는 이전 재설계의 비대칭 페널티 원칙을 “오클릭이 놓침보다 무겁다”로 재적용한 것입니다.

ARCHIVE LENS 는 활성화 시 현재 켜진 모든 신호의 남은 시간에 고정 보너스(`LENS_EXTEND_BONUS_MS`)를 더하고, 이후 4초(`LENS_BOOST_MS`) 동안 새로 켜지는 신호의 지속시간을 1.6배로 늘립니다. DEEP VERIFY 는 활성화 시점부터 5초(`DEEP_VERIFY_WINDOW_MS`) 동안 모든 판정에 2배 배율을 적용하며, 런당 한 번만 켤 수 있습니다. 두 자원 모두 실제 판정 로직(`scorePurge` / `getWrongClickLoss`)에 배율/보너스 값만 전달할 뿐, 판정 자체를 바꾸지 않습니다.

이전 버전에서 사용하던 `@chenglou/pretext` (정적 텍스트 리플로 엔진)는 SIGNAL TRIAGE 재설계 때 실사용처가 사라져 의존성을 이미 제거했으며, 이번 재설계에서도 다시 필요해지지 않았습니다.

채점 기준점도 함께 바뀌었습니다. 기존에는 “세션이 제시된 시점”부터 경과 시간을 측정했지만, 이제는 도킹된 터미널의 `deadline / totalMs` (그 세션이 처음 켜진 시점부터의 전체 잔여시간 비율)를 기준으로 `scoreCorrectAnswer` 를 호출합니다 — 이동 시간까지 포함해 “빨리 도착해 빨리 판정할수록” 보상하도록 재정의했습니다.

쿠키 상태 모델

단일 쿠키 `state_less_memory_v1` 에 다음 화이트리스트 필드만 저장합니다.

```
{
  "version": 2,
  "visits": 1,
  "runs": 0,
  "bestScore": 0,
  "lastEnding": null,
  "policy": "session",
  "fragments": 0
}
```

값은 읽은 뒤 형식·범위·허용 열거형을 다시 검증합니다. 최신 보안 컨텍스트에서는 이벤트 루프를 막지 않는 Cookie Store API를 우선 사용하고, 미지원 브라우저에서는 `SameSite=Strict` 인 `document.cookie` 로 전환합니다.

결정적 로컬 추론 (SIGNAL STRIKE 이전, 참고용)

이전 재설계(SESSION AUDIT·이동+도킹)까지는 방문·테마·시간·입력·탭·화면 폭·네트워크·이전 결말에 완료 런·보존 정책·최고 점수·기억 조각을 더한 14개 사실마다 참/거짓/오염 문장을 정의하고, 세션 시드로 이 중 몇 개를 결합해 판정 대상을 만들었습니다(`createFactCatalog` / `createSessionClaims`). SIGNAL STRIKE로 재설계하며 판정 대상 자체가 텍스트 주장에서 색·

기호 신호로 바뀌어 이 14개 사실 카탈로그와 결합 로직은 더 이상 필요하지 않아 `game-logic.js` 에서 제거했습니다. 대신 “결정적 로컬 규칙”이라는 원칙은 §4 “SIGNAL STRIKE 반응형 판정”의 세 escalation 곡선(경과 시간만의 순수 함수)으로 이어집니다.

캐릭터 이미지 프롬프트

캐릭터/MORI_기본이미지_생성_프롬프트.md 에는 초기 성인 여성 설정, 양면적 성격, 색상, 전신 구도와 파일 규격을 기록했습니다. 참여자가 선택한 1024×1536 RGBA PNG는 실투명 알파로 확인 후 전신 기준으로 확정했습니다.

추가 피드백 이후 캐릭터보다 UI 설정을 먼저 확정했습니다. docs/UI_UX_아트디렉션.md 에서 브라우저 로컬 기억 보관소의 경로·색인·보존·정정 언어를 정하고, 캐릭터/MORI_상황별_캐릭터_설계.md 에서 그 UI에 맞는 기억 사서 실루엣과 복장, 전신·흉상 규격, 17개 런타임 상태, 기존 원본을 잠근 마스킹 편집 프롬프트를 작성했습니다. AI 생성 인상을 줄이기 위해 한 번에 여러 장을 생성하지 않고, 한 기준 흉상에서 표정·손·소품만 편집한 뒤 선화와 손을 수동 정리하도록 제작 절차를 제한했습니다.

실제 17장을 제작하는 과정에서 프롬프트 문구만으로는 안 잡히는 문제가 반복해서 나타났습니다. 배경을 “투명 알파 또는 균일한 단색”으로 열어두자 생성기가 하필 검정을 골랐는데, 캐릭터의 머리색·선화도 검정에 가까워 자동으로 배경을 지우면 머리카락에 구멍이 뚫리는 위험이 있었습니다. 이후 배경 요구사항을 “투명 알파만 허용, 예시 색상 없음”으로 좁히고, 정사각형 캔버스 요구를 참조 이미지에 종속되지 않도록 별도로 못박은 뒤에야 17장 모두 실제 투명 배경·1024×1024 정사각형으로 나왔습니다. 또한 감정 상태별 색 표식(예: 정답 카드의 초록 체크, 복구 카드의 초록 스티치)이 프롬프트에 있어도 빠지는 경우가 있어, 이후 프롬프트마다 “이 요소가 안 보이면 생성 실패로 간주하고 다시 그린다”는 조건을 추가해 재현율을 높였습니다. 완성된 이미지는 관찰(observing) 기준 흉상과 겹쳐 얼굴·머리핀 좌표가 흔들리지 않는지 확인한 뒤 실제 게임에 연결했습니다.

5. 외부 에셋과 오픈소스

항목	용도	버전	라이선스·출처
Vite	개발 서버와 프로덕션 번들	8.2.0	MIT, https://github.com/vitejs/vite
OpenAI Codex	개발 보조(SESSION AUDIT까지)	사용 환경 제공 버전	https://openai.com/codex/
Claude Code (Anthropic)	개발 보조(터미널 이동+도킹 재설계, Puppeteer 검증)	사용 환경 제공 버전	https://www.anthropic.com/claude-code
puppeteer-core	재설계 검증용 헤드리스/로컬 Chrome 조작	package.json 명시 버전	Apache-2.0, https://github.com/puppeteer/puppeteer
Pandoc	제출용 PDF 생성 시 마크다운 →HTML 변환(게임 실행에는 사용하지 않음)	3.9.0	GPL-2.0-or-later, https://pandoc.org/
JetBrains Mono	일지·터미널 텍스트용 모노스페이스 폰트, <code>public/fonts/</code> 에 자체 호스팅	v24 (가변 폰트)	SIL Open Font License 1.1, https://github.com/JetBrains/JetBrainsMono
“Industrial Machine Drone (Seamless Loop)”	플레이 화면 배경 앰비언트 루프 (<code>src/assets/audio/core-drone-loop.mp3</code>)	Freesound #854269	CC0 1.0(퍼블릭 도메인), 제작자 kkenny101, https://freesound.org/s/854269/

MORI 전신 기준표와 17개 상황별 흉상 이미지는 참여자가 기록된 프롬프트를 사용해 ChatGPT Image에서 생성·선택했습니다. UI 그래픽은 HTML/CSS로 직접 만들고, 효과음(정화·오클릭·경보·콤보·결과)은 Web Audio API 오실레이터로 그 자리에서 합성합니다. 배경 앰비언트 루프 1개는 위 표의 Freesound CC0 음원을 그대로 가져와 씁니다 — 이 프로젝트는 “합성 오디오만

사용한다”는 원칙을 처음부터 못박아 두지 않았고, 라이선스가 명확하고 출처를 표에 남길 수 있는 외부 에셋이라면 필요에 따라 계속 추가할 수 있습니다.

5.1 배경 앰비언트 루프 추가(Freesound API)

“웹 검색을 통해서 보충할수있나?”(오디오·강화 다양성·메타 진행 보강 방법을 물음) 다음, 참여자가 Freesound 계정에서 발급한 API 자격증명을 전달하며 “시스템의 루트에 해당 api key 저장하고(이 폴더에는 사용여부만 기재), 작업하자, 또한 그외에 외부 에셋 추가 가능성은 열어놓자, 다른 가능성을 닫는 문서내용 전부 없애고, 작업해”라고 지시했습니다. (자격증명 원문은 보안상 이 문서에 인용하지 않습니다.)

Freesound API는 **Token 인증만으로 검색과 미리듣기(preview) 파일 다운로드가 가능**하고, 원본 파일 다운로드에만 OAuth2 로그인에 필요합니다. 게임에 쓰는 짧은 루프는 미리듣기 품질(HQ MP3, 128kbps)로도 충분해 OAuth2 없이 Token 인증만으로 전체 과정을 완료했습니다. 자격증명은 저장소에 절대 포함하지 않고, 참여자의 zsh 환경 변수(`~/.zshrc` 의 `FREESOUND_API_KEY` / `FREESOUND_CLIENT_ID`)에만 저장해 로컬 조회에 사용했고, 이 문서와 저장소에는 “Freesound API 사용함”이라는 사용 여부만 남깁니다.

검색 기준은 CC0 라이선스 필터, `seamless loop` (완전 루프) 태그, 게임 테마(신호·코어·SF 앰비언트)와의 어울림이었습니다. 최종 선택한 “Industrial Machine Drone (Seamless Loop)”(Freesound #854269, kkenny101, CC0)은 7.7초 완전 루프라 60초 런 내내 이어 붙여도 이어지는 지점이 들리지 않습니다. `src/audio.js` 의 `AudioEngine` 에 `startDrone` / `setDroneTension` / `stopDrone` 을 추가해 `AudioBufferSourceNode` 로 반복 재생하고, 로우패스 필터 컷오프를 코어 무결성에, 게인을 남은 시간 비율에 연동했습니다 — 코어가 손상될수록 소리가 먹먹해지는 것은 코어 오브가 시각적으로 흐려지는 것과 같은 상태를 청각으로 반복하는 장치입니다. 기존 효과음(정화·오클릭 등)은 전부 그대로 합성 방식을 유지하고, 이 배경 루프만 외부 음원으로 대체했습니다.

6. 검증 명령

```
npm test
npm run build
STATELESS_GAME_URL=http://127.0.0.1:5173/ STATELESS_INPUT_MODE=keyboard npm run record
STATELESS_GAME_URL=http://127.0.0.1:5173/ STATELESS_INPUT_MODE=mouse npm run record
npm run docs:pdf
```

`npm run docs:pdf (scripts/build-docs-pdf.mjs)` 는 로컬 Pandoc으로 `docs/게임_소개.md` / `docs/AI_활용_기술.md` 를 `pdf-style.css` 가 적용된 HTML로 변환한 뒤, 로컬 Chrome의 `page.pdf()` 로 A4 PDF를 생성합니다. 생성 후에는 매번 `pdftoppm` 으로 각 페이지를 이미지로 렌더해 잘림·빈 페이지·깨진 한글이 없는지 직접 확인합니다.

`npm test` 는 20개 게임 규칙·쿠키 직렬화 테스트를 실행합니다. `npm run build` 는 GitHub Pages에 배포할 `dist/` 정적 파일을 생성합니다.

`scripts/record-game.mjs (npm run record)` 는 이전 SIGNAL LOCK 화면 구조(`#statement-board`, `.statement-chip` 등)를 자동으로 조작하도록 작성되어, SIGNAL TRIAGE 재설계로 해당 DOM 요소가 제거되면서 이미 실행할 수 없는 상태였습니다. 이후 이동+도킹 재설계(세션 카드 → 터미널 룸)와 SIGNAL STRIKE 재설계(터미널 룸 → 신호 슬롯 6개)를 거치며 화면 구조가 두 번 더 바뀌어 여전히 실행할 수 없습니다. 세 번의 재설계 모두 검증은 이번 문서에 기록된 임시 Puppeteer 스모크 테스트로 대체했으며, 녹화 스크립트를 현재 화면 구조에 맞게 다시 쓰는 작업은 진행하지 않았습니다 — 화면 구조가 이렇게 여러 번 바뀌는 개발 단계에서는 매번 녹화 스크립트를 유지보수하는 비용이 임시 스모크 테스트를 새로 짜는 비용보다 크다고 판단했습니다. 제출용 플레이 영상은 이 자동 녹화 대신 실제 사람이 화면을 직접 녹화합니다(제출_체크리스트.md 참고).